ZombieVerter VCU

Training Series

Complete reference guide for DIY EV conversion builders

28 modules · 6 tracks · Firmware V2.30A (August 2025)
github.com/robertwa1974/zombieverter-training
openinverter.org · evbmw.com

F Foundation	H Hardware	W Wiring	C Configuration	I Integration	A Advanced
F01–F04	H01	W01–W06	C00–C06	I01–I04	A01–A06

Table of Contents

TRACK FOUNDATION · MODULE F01

What Is a VCU and Why Do You Need One

The Problem With Donor EV Components

When you salvage an inverter and motor from a wrecked Nissan Leaf, you have powerful, proven hardware — but that hardware was designed to talk to a Leaf BMS, a Leaf dashboard, a Leaf brake controller, and a Leaf battery management system. None of which you have.

Every donor EV component speaks its own proprietary CAN protocol. The inverter expects specific CAN messages on specific message IDs at specific baud rates. It won't spin without them. Building a custom controller to reverse-engineer and replicate those messages for every possible inverter type would take months per build.

A Vehicle Control Unit solves this problem. It's a purpose-built translator that sits between your donor drivetrain components and your conversion vehicle, speaking the language each component needs.

What a VCU Does

In a ZombieVerter-based conversion, the VCU is responsible for:

Contactor sequencing — closing the negative contactor, running precharge through a resistor, monitoring HV bus voltage, then closing the main contactor in the correct sequence. This protects the inverter capacitors from inrush current damage on every startup.

Torque control — reading the throttle pedal, mapping it to a 0–100% demand, and sending the appropriate CAN torque command to the inverter. Without this, the inverter has no way to know how much torque the driver is requesting.

Direction control — handling forward/reverse selection and communicating motor direction to the inverter.

Regen control — managing regenerative braking via throttle lift and brake pedal, sending negative torque commands to the inverter (V2.15A onwards).

Charge control — managing the onboard charger and DC-DC converter via CAN when plugged in.

BMS integration — reading current limits from a cell BMS and enforcing them as caps on motor torque and charge current.

Vehicle integration — sending spoofed CAN messages to keep the OEM instrument cluster functional (speedometer, tacho, warning lights).

Safety — opening contactors immediately on fault conditions, HVIL loop monitoring, brake interlock enforcement.

What a VCU Is Not

A VCU is not a motor controller or inverter. It does not switch the high-voltage phases to the motor. The inverter does that. The VCU tells the inverter what torque to produce — the inverter does the actual power switching.

A VCU is not a BMS. It does not monitor individual cell voltages or temperatures, balance cells, or protect against cell-level faults. That is the job of a separate BMS (SimpBMS, Orion, etc.). The VCU reads the output of the BMS — current limits — and enforces them.

What Makes ZombieVerter Different

Before the ZombieVerter, DIY EV converters had two options: write custom Arduino/microcontroller code to replicate inverter CAN messages (high skill barrier, per-inverter effort), or use the openinverter framework on a custom PCB (powerful but complex to set up).

The ZombieVerter packages the openinverter framework in a purpose-built VCU board with:

- **No programming required** — all configuration through a browser-based web interface over Wi-Fi
- **Multi-inverter support** — select your inverter type from a dropdown; the VCU handles the rest
- **Multi-charger support** — Leaf PDM, VW OBC, Tesla Gen2/3, Outlander OBC, CHAdeMO, CCS
- **IO Matrix** — configurable input/output pin functions without code changes
- **Active community** — openinverter.org forum with hundreds of documented builds

---

The Hardware

The ZombieVerter board is built around two processors:

STM32F107 — the main MCU. Runs the VCU firmware, manages all I/O, handles CAN communication with all connected devices, runs the contactor sequencing logic, calculates torque demands.

ESP8266 — the Wi-Fi module. Hosts the web configuration interface. Broadcasts a Wi-Fi access point you connect to from any browser. Does not run VCU functions — it's purely the configuration layer.

The board connects to your vehicle through a 56-pin Aptiv automotive connector. All vehicle wiring — power, signals, CAN buses, contactor outputs — goes through this one connector.

> Available from evbmw.com as a complete pre-programmed kit, partial kit, or PCB-only.

The Community

The ZombieVerter is designed and maintained by Damien Maguire (EVBMW) and built on Johannes Huebner's openinverter firmware framework. It is genuinely open source — schematics, firmware, and board files are on GitHub.

The community is at openinverter.org/forum. Hundreds of completed builds are documented there, covering BMW E-series, Volkswagen, Mitsubishi, classic vehicles, and many others.

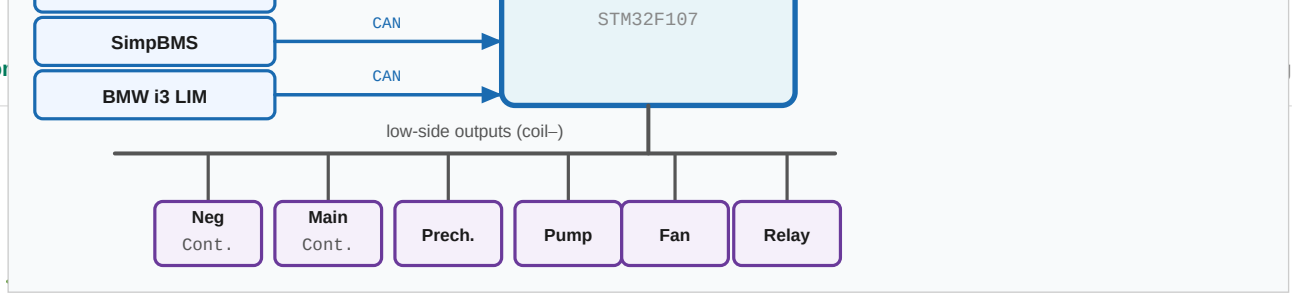
Damien's YouTube channel (@Evbmw) documents everything he builds and every firmware release with walkthrough videos — an invaluable resource that directly informed this training series.

Next Steps

- **F02** — ZombieVerter Ecosystem Overview: supported hardware, community resources
- **H01** — VCU Hardware Walkthrough: the board in detail
- **C03** — Essential Parameters: First Start: get to run mode on your bench

Source: Damien Maguire @Evbmw — "New VCU Project Introduction" (February 2021)

Last verified against firmware: V2.30A (August 2025)



mbieVerter Ecosystem Overview — wiring overview

What Is the ZombieVerter?

The ZombieVerter is an open-source Vehicle Control Unit (VCU) designed specifically for DIY electric vehicle conversions. It was created by Damien Maguire (EVBMW) with contributions from the broader openinverter.org community.

Its core purpose is to solve a real problem: modern EV conversion projects often reuse salvaged components — motors, inverters, chargers, battery packs — from wrecked or retired EVs. Each of these components uses different control signals and communication protocols. Building a custom controller for every possible combination would be impractical. The ZombieVerter instead provides a general-purpose VCU with a wide variety of inputs, outputs, and pre-built support for the most common donor vehicle components.

In short: **the ZombieVerter is the brain that makes mismatched salvaged EV parts work together.**

The People Behind It

Person / Group	Role
Damien Maguire (EVBMW)	Primary hardware designer, firmware lead, project creator
Johannes Huebner (johu)	Creator of the underlying openinverter firmware framework the VCU builds on
openinverter.org community	Ongoing firmware contributions, inverter support classes, testing, documentation

The project is genuinely open-source. Schematics and firmware are available on GitHub. PCB design files for the current production revision (V1.a) are available through Damien's Patreon at the Design Files tier.

Where the Community Lives

Resource	URL	What's There
Main wiki	openinverter.org/wiki	Hardware docs, wiring guides, parameter references
Forum	openinverter.org/forum	Support threads, development discussion, build logs
GitHub (firmware)	github.com/damienmaguire/Stm32-vcu	Source code, firmware releases
GitHub (hardware V1)	github.com/damienmaguire/Stm32-vcu/tree/master/Hardware/Zombie	PCB files (V1 only)
EVBMW Webshop	evbmw.com	Boards for purchase
Damien's YouTube	youtube.com/@Evbmw	Build videos, firmware walkthroughs
Patreon	patreon.com/evbmw	PCB design files (V1.a), early access content

> **Note:** The forum is the single best source for troubleshooting. Most problems have been encountered and solved by someone before. Search before posting.

Hardware Features

The ZombieVerter VCU is built around an STM32F107 microcontroller (or GD32F107 on some early pandemic-era boards — see Module C01 for the distinction). An ESP8266 Wi-Fi module provides the wireless interface.

- On-board Wi-Fi (ESP8266)
- 3× CAN bus interfaces
- LIN bus
- Synchronous serial interface (used for GS450h, GS300h, Prius)
- OBD-II interface
- 3× high-side PWM driver outputs
- 5× low-side outputs
- 3× ground-pull input pins
- Connector (cable side): Aptiv 56-pin connector
- Header (board side): Aptiv 56-pin header
- Terminal removal tool: Aptiv/Delphi Part No. 210S048

Software Features

All configuration is done through a browser-based web interface — no laptop software to install, no USB cables required for normal operation. You connect to the VCU's Wi-Fi access point and navigate to its IP address in any browser.

- Contactor sequencing (negative, positive, precharge)
- Charger control and charge timers
- Motor/inverter control (torque requests via CAN or serial)
- Heater control
- Water pump and coolant fan control
- Throttle mapping and regen
- BMS limits (charge/discharge current limiting)
- ISA IVT shunt initialization and monitoring
- Data logging and live graphing

Currently Supported Hardware

> *This list grows with each firmware release. Always check the wiki for the latest.*

Motors / Drive Units

Component	Interface
Nissan Leaf Gen1/2/3 inverter/motor	CAN (180V minimum)
Lexus GS450h inverter / L110 gearbox	Sync serial
Lexus GS300h inverter / L210 gearbox	Sync serial

Component	Interface
Toyota Prius/Yaris/Auris Gen3	Sync serial
Mitsubishi Outlander front/rear motors	CAN
OpenInverter controller board	CAN

Chargers / DC-DC Converters

Component	Notes
Nissan Leaf PDM (Gen1, 2, 3)	Charger + DC-DC combined unit
Mitsubishi Outlander OBC	Charger + DC-DC combined unit
Tesla Model S DC-DC converter	
BMW i3 LIM module	Enables CCS DC fast charging (Type 1 & Type 2)
CHAdeMO DC fast charging	Via internal CAN
Foccci CCS controller	
Elcon charger	

Heaters

Component	Interface
VAG/VW PTC water heater	LIN bus
VAG/VW cabin heater	LIN bus
Opel Ampera / Chevy Volt 6.5kW heater	
Mitsubishi Outlander water heater	

BMS / Battery Monitoring

Component	Notes
ISA IVT shunt	Primary current/voltage monitoring method
Nissan Leaf ZE1 BMS / battery pack	
Renault Kangoo 36 BMS	
Orion BMS	
`SimpBMS`	
BMW S-BOX	
VW E-BOX	

> **Important:** The ZombieVerter does **not** implement direct BMS cell-level communication. It uses the ISA shunt for pack-level monitoring (voltage, current, state of charge). Cell-level BMS functions (balancing, per-cell voltage/temp) are handled by a separate BMS device such as SimpBMS, which communicates with the VCU via CAN.

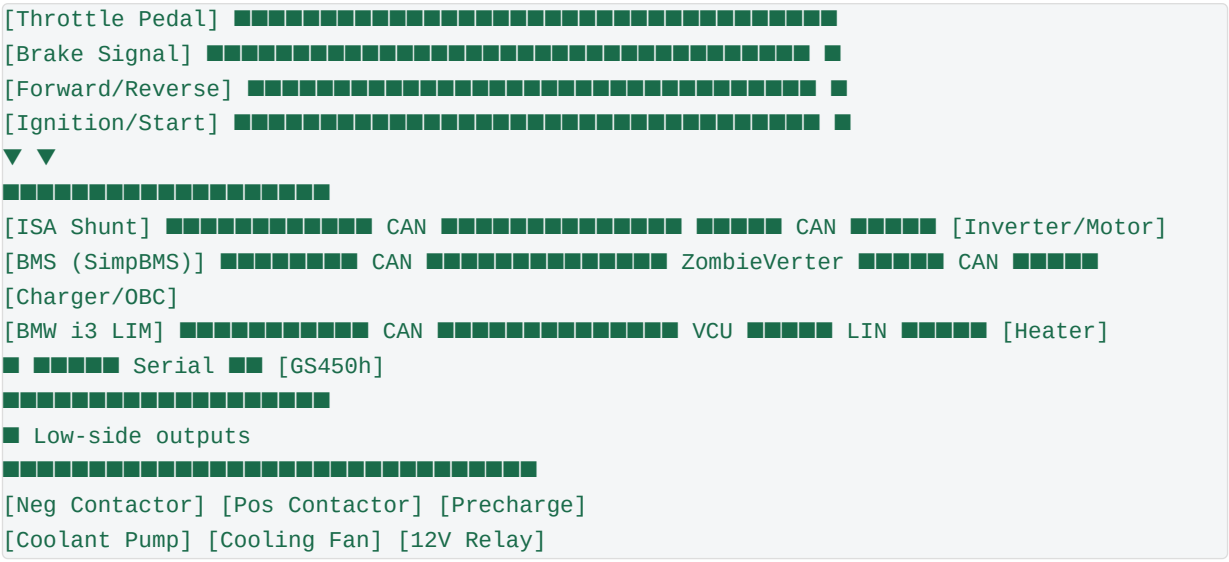
Vehicle CAN Integration (Dashboard / Instrument Cluster)

Vehicle	Notes
BMW E46 (1998–2005 3-series)	CAN + digital IO

Vehicle	Notes
BMW E39 (1996–2003 5-series)	CAN + digital IO
BMW E65 (2001–2008 7-series)	CAN
BMW E9x	CAN
Mid-2000s VAG vehicles	CAN
Subaru	CAN

How the VCU Fits Into a Conversion

A typical ZombieVerter-based conversion looks like this:



The VCU is the central coordinator. It reads driver inputs, monitors pack state via the shunt, sequences the contactors, and sends torque commands to the inverter — all configurable through the web interface without writing a line of code.

Getting a ZombieVerter Board

There are two ways to obtain a board:

- Fully-built (tested, programmed, no soldering required) — recommended for most users
- Partially-built kit (some assembly required)
- Lead time: up to 4 weeks from order
- PCB design files for V1 are on GitHub (free)
- PCB design files for the current V1.a revision require Patreon membership
- You will need to source and solder components yourself

Note: Boards from the webshop come **pre-programmed** and ready to use. You do not need to flash firmware before first use.

What's Next

- **H01** — VCU Hardware Walkthrough: a physical tour of the board, connectors, and indicators

- **C01** — Flashing Firmware: only needed if you have a blank board or need to recover a bricked unit
- **C02** — The Web Interface Walkthrough: connecting to the VCU and navigating the UI

Source: openinverter.org/wiki/ZombieVerter_VCU | openinverter.org community

Last verified against firmware: V2.30A (August 2025)

TRACK FOUNDATION · MODULE F03

EV Drivetrain Fundamentals

Two Voltage Domains

Every EV conversion has two completely separate electrical systems that must never be directly connected to each other:

High Voltage (HV) domain — the traction system. Battery pack, inverter, motor, onboard charger, DC-DC converter input. Typically 200–450V DC in ZombieVerter-based conversions. Orange cables by convention. Fatal at the current levels available.

Low Voltage (LV) domain — the vehicle electronics. 12V battery, VCU, lights, accessories, sensors, CAN buses, instrument cluster. Conventional automotive 12V. The VCU lives entirely in the LV domain — it controls the HV domain through contactors and CAN commands, but never carries HV itself.

The DC-DC converter bridges these domains in one direction only: it takes HV input and produces 12V output to charge the LV battery. Current never flows the other way.

The Traction Chain

From driver input to wheel torque, the path is:

```
Throttle pedal → VCU (torque demand) → Inverter (AC phase switching) → Motor → Gearbox/wheels
```

Throttle pedal — generates a 0–5V analog signal (Hall effect or potentiometer). The VCU reads this and maps it to a 0–100% torque demand.

VCU — translates driver demand to a CAN torque command in the inverter's protocol. Also enforces limits from the BMS, manages regen, handles direction.

Inverter — takes DC from the HV battery and switches it as three-phase AC to the motor. Switches at high frequency (typically 10–15 kHz) to produce sinusoidal current. Controls motor torque precisely by controlling the phase currents.

Motor — converts electrical energy to mechanical torque. In hybrid-derived systems (Leaf, GS450h, Prius, Outlander) the motor is a permanent magnet AC machine — efficient, high power density, well-suited for EV use.

Resolver or encoder — the motor position sensor. Tells the inverter exactly where the rotor is so it can switch phases at the right time. Without resolver feedback, the inverter cannot control the motor. The resolver cables run from the motor back to the inverter — these are the fragile signal cables that must be treated carefully.

Motor Types You'll Encounter

Permanent Magnet AC (PMAC / IPMSM) — used in Nissan Leaf (EM57, EM61), Mitsubishi Outlander, most modern EV traction motors. High efficiency, high torque density. Requires resolver feedback. The ZombieVerter supports several of these via CAN.

Induction (AC) — used in older Tesla Model S/X. Lower peak efficiency than PMAC but simpler magnetics. Supported via the openinverter board.

Dual motor (Power split) — the GS450h and Prius use two motor-generators (MG1 and MG2) with a planetary gearset between them. In EV conversion use MG2 provides primary traction, MG1 provides additional torque when the input shaft is locked. See Module I03 for detail.

Contactor Logic and Precharge

The HV battery cannot be connected directly to the inverter. The inverter contains large DC bus capacitors — connecting HV to uncharged capacitors causes an instantaneous high-current surge that can weld the contactor contacts permanently closed or destroy the capacitors.

The solution is precharge: a resistor in series with the positive HV rail that limits inrush current while the capacitors charge slowly.

1. Negative contactor closes (HV– connected to inverter)
2. Precharge contactor closes (HV+ through resistor to inverter)
3. Capacitors charge slowly — bus voltage rises over ~2–5 seconds
4. When bus voltage reaches threshold (`udcsw`), main positive contactor closes
5. Precharge contactor opens (resistor bypassed)
6. System enters run mode — full HV available

The VCU manages this entire sequence. The ISA shunt or equivalent sensor provides the bus voltage reading that tells the VCU when precharge is complete.

> **Why this matters for first-time builders:** `udcsw` defaults to 330V. With no HV battery connected on the bench, the bus voltage never reaches 330V. Precharge appears to fail repeatedly. The fix is to set `udcsw` to 0 or 15V for bench testing. See Module C03.

Regenerative Braking

Regen reverses the energy flow. When the driver lifts the throttle or presses the brake, the VCU sends a negative torque command to the inverter. The inverter runs the motor as a generator, converting kinetic energy back to DC electricity and pushing it back into the battery.

The amount of regen is controlled by:

- `throtmin` — negative value = throttle-lift regen (e.g. -5 = light regen when foot off throttle)
- Brake input — `din_brake` can trigger regen in addition to friction braking
- BMS current limits — the BMS caps how much current can flow back into the battery

> Regen was first reliably implemented in V2.15A (April 2024). Content about regen from before this date refers to a non-functional or experimental implementation.

The ISA Shunt — Pack Monitoring

The VCU cannot directly measure HV bus voltage — it operates entirely in the 12V domain. An external sensor bridges this gap.

The ISA IVT-S shunt sits in series with the HV bus. It measures voltage, current, and calculates state of charge, then transmits these readings over CAN to the VCU. The VCU uses this data to:

- Monitor precharge progress (is the bus voltage rising?)
- Know when precharge is complete (has it reached `udcsw`?)
- Enforce discharge and charge current limits
- Report pack state to the driver

The ISA shunt must have permanent 12V power — the same always-on feed as the VCU. If powered from ignition-switched supply it loses its coulomb count state when the key is off, and the next startup may read stale or zero UDC.

Isolation and Grounding

The HV system must be electrically isolated from the vehicle chassis (LV ground). This isolation is safety-critical — if HV+ or HV− contact the chassis, the chassis becomes energised at HV potential.

The ISA shunt measures the voltage difference between HV+ and HV− without either being referenced to chassis ground. The inverter is similarly isolated. The battery pack enclosure may be grounded to chassis for EMC reasons but the battery terminals must not be.

HVIL (High Voltage Interlock Loop) is a low-current 12V loop that passes through every HV connector in the system. If any HV connector is opened or a cable is disconnected, the loop breaks, the VCU detects it and immediately opens the contactors. See Module W04.

Source: Damien Maguire @Evbmw — "Lexus GS450h EV Drivetrain Deep Dive" (June 2020)

Last verified against firmware: V2.30A (August 2025)

TRACK FOUNDATION · MODULE F04

High Voltage Safety

This module is not optional. High voltage EV systems can kill instantly, silently, and without warning. Read this before touching any orange cable, any contactor, or any HV terminal.

The Risk

A typical EV conversion pack runs at 200–450V DC. At these voltages, as little as 100mA of current through the body is sufficient to cause fatal ventricular fibrillation. The current available from a traction pack is measured in hundreds of amps — far beyond what causes death.

The danger is compounded because:

- **DC does not let go.** AC at 50/60 Hz causes muscle contractions that may allow you to pull away. DC causes sustained muscle contraction — if you grip a live conductor your hand locks closed.
- **The capacitors retain charge.** After contactors open, the inverter bus capacitors remain charged at near-pack voltage for minutes. The pack is isolated but the inverter is still dangerous.
- **Contactors can fail closed.** A welded contactor leaves the HV system live even when the VCU believes it is off.

The Two-Person Rule

Never work on a live HV system alone.

If something goes wrong — a shock, a fall, a fault — you need someone present who can kill the power and call for help. This is not a guideline. It is the single most important safety practice in this entire module.

Personal Protective Equipment

- **Insulated gloves rated for the voltage** — Class 0 (1000V rated) or higher. Not rubber washing-up gloves, not latex medical gloves. Proper electrical insulating gloves from a safety supplier. Inspect before every use — a pinhole is invisible and fatal.
- **Safety glasses** — arc flash and battery acid protection.
- **No jewellery** — rings, watches, and chains are conductors. Remove them entirely before working near any electrical system, HV or LV.
- Face shield
- Non-conductive tools
- Insulated floor mat

The Discharge Procedure

Before working on any HV component — inverter, charger, battery, contactor box — verify the system is de-energised:

1. **Key off, key removed from vehicle**
2. **Disconnect the 12V auxiliary battery** — this removes power from the VCU and prevents contactor actuation

3. **Open the HV service disconnect** — every EV pack should have a manual service disconnect (MSD) that physically breaks the HV circuit. Pull it, pocket it.
4. **Wait minimum 5 minutes** — allows inverter capacitors to discharge through bleed resistors. Some inverters take longer. Check your inverter specification.
5. **Verify with a multimeter** — set to DC voltage, highest range. Measure across HV+ and HV– at the inverter DC bus terminals. Should read near zero. If it reads anything above 50V, wait longer and measure again.
6. **Only then touch any HV component**

Never trust the contactors alone. Contactors can fail closed (welded). The 12V interlock can fail. The VCU can malfunction. The only safe assumption is that the HV system is live until you have personally measured otherwise with a calibrated meter.

Fusing Philosophy

Every HV conductor must be protected by a fuse rated for the maximum fault current. For a ZombieVerter-based build:

HVIL — High Voltage Interlock Loop

HVIL is a low-current 12V loop that passes through every HV connector, service plug, and access panel in the system. If any connector is opened or any panel is removed, the loop opens, the VCU detects the break and immediately opens the contactors.

This means:

- Unplugging an HV cable without discharging first triggers a contactor open, not electrocution
- Removing a battery lid triggers a contactor open
- A failed HV connector triggers a contactor open

HVIL does not replace the discharge procedure — contactors are mechanical devices and can fail. But it dramatically reduces the probability of accidental energisation during service.

The BMW Safety Box (S-BOX) integrates HVIL with pack-level CAN monitoring in one unit. See Module W04 for HVIL wiring.

Cable Colour Conventions

Colour	Meaning
Orange	HV cables — always treat as potentially live
Red	12V positive
Black	12V negative / chassis ground
Blue	Neutral (some charger AC wiring)
Brown	Line (some charger AC wiring)

When working on an unfamiliar vehicle, treat all orange cables as live regardless of the system state you believe it to be in.

Bench Testing Safety

Most first-start testing is done on 12V only — no HV battery connected. This is intentionally the safest way to verify wiring and parameters before introducing HV risk.

When you do connect HV for the first time:

- Use a current-limited bench supply set to a low voltage (60V, 2A) before using full pack voltage
- Start with a bench precharge verification — watch `udc` rise in Spot Values before attempting run mode
- Have the 12V auxiliary battery disconnect within reach before and during the first HV test
- Verify contactor sequencing on 12V before connecting any HV cables

---

If Someone Is Shocked

1. **Do not touch them** — if they are still in contact with a live conductor, you will be shocked too
2. **Kill the power** — pull the service disconnect, disconnect the 12V battery, use a non-conductive object to break the contact if you cannot reach the disconnect
3. **Call emergency services immediately** — even if the person appears conscious and uninjured, cardiac arrest can occur up to 24 hours after an electrical injury
4. **CPR if necessary** — electrical shock can cause cardiac arrest without obvious external injury

Sources: openinverter.org community practice · BMW Safety Box series — Good Enuff Garage (April–May 2024) · general electrical safety standards

Last verified: August 2025

TRACK HARDWARE · MODULE H01

VCU Hardware Walkthrough

Board Overview

The ZombieVerter VCU is a compact single-board controller built around two main chips:

- **STM32F107** — the main microcontroller, running the VCU firmware
- **ESP8266** — a plug-in Wi-Fi module that hosts the web interface

> **GD32F107 Warning:** A small number of boards produced during the 2021–2022 semiconductor shortage used a GigaDevices GD32F107 chip instead of the STM32. These "GD boards" require a separate firmware branch that has since been abandoned and diverged significantly from the main codebase. If you have an older board, check your chip markings before flashing. See Module C01 for details.

The current production hardware revision is **V1.a**. An earlier V1 (pre-production) revision exists — the main wiki image labels it V1.2 to distinguish it. V1.a is the board you will receive from the webshop today.

Physical Layout

The board is housed in an IP-rated enclosure with a 56-pin Aptiv connector on one end. Key physical features:

The 56-Pin Main Connector

This is the single point of connection for all vehicle wiring. All power, ground, signal inputs, and outputs pass through this connector. It uses Aptiv (formerly Delphi) automotive-grade terminals.

Part	Aptiv Part Number
Cable-side connector housing	Aptiv 56-pin connector
Board-side header	Aptiv 56-pin header
Terminal removal tool	210S048 (Aptiv/Delphi)

- Full enclosure kit with header, connector, and pins
- Connector and pins only
- Pre-wired connector harness (most convenient for initial bench testing)

The Wi-Fi Module

The ESP8266 plugs into a socket on the main board. It can be removed — this is sometimes recommended when doing an ST-Link firmware recovery, to avoid any interference during the flash process.

> **Known issue with newer boards:** Some recent units have a new Wi-Fi module variant that does not automatically assign an IP address via DHCP. If you connect to the Wi-Fi but cannot reach 192.168.4.1, manually set your computer's IP address to `192.168.4.2` (or any address in the 192.168.4.x range other than .4.1). See the forum thread linked in the wiki for ongoing status.

Status LEDs

The board has several indicator LEDs:

LED	Label	Meaning
Activity	acty	Should be flashing when the board has 12V power. A steady (non-flashing) light or no light indicates a problem.
ESP8266 LEDs	on Wi-Fi module	Indicate Wi-Fi module status

If `acty` is not flashing after applying 12V power, the board is not running correctly. Common causes: insufficient power supply, bad ground connection, or a firmware issue requiring recovery.

SWD Test Points (C, G, D)

Three test points on the PCB are used for ST-Link firmware recovery:

- **C** — SWClk (clock)
- **G** — Ground
- **D** — SWDio (data)

These are only needed if the board is bricked and cannot be recovered via the web interface. See Module C01.

Power Requirements

The ZombieVerter requires a **permanent 12V supply** — not ignition-switched. This is intentional: the VCU needs to remain powered when the vehicle is parked to manage charge timers, monitor systems, and handle charging sessions.

Parameter	Value
Supply voltage	12V DC
Idle current draw	~150 mA
Power pin	Pin 56 (12V+)
Ground pin	Pin 55 (12V–)

150 mA at idle is modest — comparable to a small relay module. With a healthy 12V battery, this will not cause meaningful parasitic drain over days of parking.

> **Do not use an ignition-switched supply for the main VCU power.** If you do, the VCU will lose its UDC (pack voltage) history when the key is off, which can cause precharge failures on the next startup. See Module W03.

Complete Pin Reference

The following table covers all 56 pins on the main connector. Fixed-function pins are listed first, followed by the configurable I/O section.

Fixed Power & Signal Pins

Pin	Function	Direction	Notes
55	12V Ground	PWR	Permanent ground
56	12V Positive	PWR	Permanent 12V — NOT ignition switched

Pin	Function	Direction	Notes
32	Ignition relay control	Output (low-side)	Pulls ground pin of 12V relay to ground — relay switches 12V "ignition" to inverter/charger/DCDC
15	Ignition ON input	Input	Pull HIGH (to 12V+) with ignition switch in ON position
52	Start input	Input	Pull HIGH momentarily with ignition switch in START position
53	Reverse input	Input	Pull HIGH (to 12V+) when reverse selected
54	Forward input	Input	Pull HIGH (to 12V+) when forward selected
49	Brake input	Input	Pull HIGH (to 12V+) when brake pedal pressed
45	Throttle ground	PWR	Ground reference for throttle pot(s)
46	Throttle channel 2	Analog input	Second pot wiper (dual-channel throttle)
47	Throttle channel 1	Analog input	First pot wiper
48	Throttle 5V supply	PWR	5V supply for throttle pot(s)
31	Negative contactor	Output (low-side)	Pulls negative contactor coil ground
33	Positive contactor	Output (low-side)	Pulls positive contactor coil ground
34	Precharge contactor	Output (low-side)	Pulls precharge contactor coil ground

Contactor wiring rule: All contactor output pins are **low-side switching** — they pull the ground leg of the contactor coil to ground. The positive leg of each contactor coil must be connected to 12V+. Do NOT connect 12V+ to these output pins.

Configurable I/O Pins

These pins can be assigned to different functions through the web interface. However, the hardware type of each pin is fixed — software cannot change a low-side output into a high-side output, for example.

Low-Side Outputs

These pull to ground when activated. Use them to switch relay coils (coolant pumps, fans, etc.). **Not suitable for directly driving loads.**

Pin assignment name	Default use
GP Out 3	General purpose
GP Out 2	General purpose
Neg Contactor switch / GP Out 1	Negative contactor (or general purpose)

Pin assignment name	Default use
Trans SL1–	GS450h transmission solenoid 1 (if not using GS450h, available as GP)
Trans SL2–	GS450h transmission solenoid 2 (if not using GS450h, available as GP)

High-Side PWM Outputs

These supply 12V+ when activated, with optional PWM modulation. Use for gauges, PWM-controlled fans, or indicator signals. **Not suitable for controlling relay coils** — use low-side outputs for relays.

Pin assignment name	Default use
PWM 1	General purpose PWM
PWM 2	General purpose PWM
PWM 3	General purpose PWM
Pump PWM	GS450h oil pump PWM, or tachometer PWM output

Ground-Pull Digital Inputs

These pins detect when pulled to ground. **Do not apply any voltage to these pins** — they are ground-detect only, not 12V-tolerant inputs.

Pin assignment name
`PB1`
`PB2`
`PB3`

Configurable Pin Functions (Software-Assigned)

The following functions can be assigned to the configurable I/O pins via the web interface. Note the type constraint — a function expecting a low-side output cannot be assigned to a PWM pin.

Function Name	Type	Description
`ChaDemoAIw`	OUTPUT	Activates when CHAdeMO charger handshake initiates
`OBCEnable`	OUTPUT	Activates as part of the ExtCharger module
`HeaterEnable`	OUTPUT	Activates in run mode when coolant pump is on
`RunIndication`	OUTPUT	Activates when VCU is in run mode
`WarnIndication`	OUTPUT	Activates when an error occurs
`CoolantPump`	OUTPUT	Activates during precharge and run
`NegContactor`	OUTPUT	Activates when negative contactor should close
`BrakeLight`	OUTPUT	Activates when brake light threshold met

Function Name	Type	Description
`ReverseLight`	OUTPUT	Activates when reverse is selected
`CoolingFan`	OUTPUT	Activates when FanTemp setpoint is reached
`HVActive`	OUTPUT	Activates when contactors are closed in run or charge mode
`BrakeVacPump`	DIGITAL OUTPUT	Activates when BrakeVacSensor threshold is met
`CpSpoof`	PWM OUTPUT	Spoofs CP signal to OBC (used with FOCCCI or i3 LIM)
`GS450Hpump`	PWM OUTPUT	Controls GS450h oil pump speed
`HeatReq`	DIGITAL INPUT	Heater request signal
`HVRequest`	DIGITAL INPUT	**Currently non-functioning**
`DCFCRequest`	DIGITAL INPUT	CHAdeMO charge interface enable
`ProxPilot`	ANALOGUE INPUT	Detects J1772 charge cable insertion
`BrakeVacSensor`	ANALOGUE INPUT	Vacuum sensor for brake booster pump control
`Switch_NoRegen`	DIGITAL INPUT	Inhibits regen (useful with manual transmission) — check firmware release notes for availability

Critical warning: The web interface will allow you to assign a function to a pin type it is not compatible with (e.g. assigning an output function to an input pin). The software will accept the assignment but the hardware will not respond. Always check that your assignment matches the physical pin type. A common mistake is wiring an input switch but assigning it to a function that requires an output.

The Proximity Pilot (ProxPilot) Circuit

The ProxPilot analogue input is used to detect when a J1772 charge cable is plugged in, which triggers the VCU to enter charge mode. This is required for chargers like the Mitsubishi Outlander OBC that don't have their own cable detection logic. (The Nissan Leaf PDM handles this internally, so ProxPilot is not needed for Leaf stacks.)

The detection circuit requires two external resistors:

```
5V (Pin 48)
■
330Ω ← R_pullup
■
■■■■■■■■■■■■■■■■ ProxPilot analogue input pin
■
R5 ← value depends on connector type:
■ Type 1 (J1772): 2.7kΩ
■ Type 2 (Mennekes): 4.7kΩ
■
GND
```

Note: The charge port itself may already contain this resistor — check before adding an external one.

1. Assign ProxPilot function to the chosen analogue input pin in the web UI

2. Monitor the `PPVal` spot value while plugging in a cable
3. Set `PPthreshold` to a value just above the reading seen when the cable is inserted
4. Be aware that the exact resistance (and therefore `PPVal`) will vary depending on the cable's rated current capacity

---

What's Next

- **H02** — Inputs & Outputs Deep Dive: electrical characteristics, voltage limits, current ratings
- **H03** — CAN Bus on the VCU: which CAN bus is which, termination, and baud rates
- **W01** — Wiring Philosophy & Best Practices: before you touch a wire

Source: openinverter.org/wiki/ZombieVerter_VCU | openinverter.org/wiki/ZombieVerter_IO

Last verified against firmware: V2.30A (August 2025)

TRACK WIRING · MODULE W01

Wiring Philosophy & Best Practices

The Philosophy

A ZombieVerter wiring harness is not a single entity — it is three logically separate fuse blocks, each with a distinct purpose, feeding different groups of components. Understanding this structure before crimping a single terminal will save hours of debugging later.

The three groups are:

1. **Permanent power** — always live, even when the vehicle is parked
2. **Ignition ON** — live when the key is in the ON position
3. **Start position** — momentary signal only, like turning a key to START

Each group has its own fuse block. Each fuse block has its own feed from the 12V battery. Mixing these groups — feeding an ignition-switched device from permanent power, or running too much current through the ignition switch — is the most common source of mysterious electrical gremlins in EV conversions.

Fuse Block Strategy

Permanent Power Block

Ignition ON Block

Start Position Block

Wire Gauging

Wire gauge is specified in AWG in North America, metric cross-section (mm²) in Europe. Higher AWG number = thinner wire — the opposite of the metric system.

Application	Suggested gauge
HV main traction cables	35–70mm² (0 AWG or larger)
HV charger / DC-DC	10–16mm² (6–4 AWG)
12V main battery feed	6–10mm² (10–8 AWG)
12V fuse block feeds	4mm² (12 AWG)
Signal wires (VCU pins)	0.5–1mm² (20–18 AWG)
CAN bus	0.5mm² twisted pair (22 AWG)

Rule: Rule: Size the wire for the load current. Then size the fuse just below the wire's current rating. The fuse protects the wire — the wire must already be sized to handle the device.

Connector Selection

Labelling

Label every wire at both ends before installation. Once a harness is routed and cable-tied, tracing an unlabelled wire is extremely time-consuming.

A simple label convention:

```
P56-12VPERM ← Pin 56, permanent 12V
P15-IGNON ← Pin 15, ignition ON signal
CAN1H-INV ← CAN1 High, to inverter
```

Use heat-shrink label sleeves or a label maker with vinyl tape. Marker pen on tape fades in heat and oil.

Dupont Cables — For Bench Only

Dupont (breadboard) jumper cables are useful for initial bench testing — they let you power up the board without committing to a permanent header. However they are loose, unreliable, and not rated for automotive vibration. Remove and replace with proper crimped terminals before the vehicle goes on the road.

Building the Harness — The Sequence

Good Enuff Garage's wire harness series (April–June 2024) documents the complete ZombieVerter harness build sequence. The recommended order:

1. **Permanent power fuse block** — build first, verify VCU powers up and Wi-Fi appears
2. **Ignition switch to VCU** — add ignition ON (Pin 15) and momentary start (Pin 52)
3. **Verify run mode on bench** — 12V only, no HV — before adding anything else
4. **Forward/reverse switch** — add direction control (Pins 53/54), verify din_forward/din_reverse in Spot Values
5. **Brake signal** — add brake switch (Pin 49), verify din_brake goes on/off correctly
6. **Throttle wiring** — add pot signals, calibrate
7. **Inverter relay** — add Pin 32 output relay, test switched supply
8. **Contactor wiring** — add precharge, negative, main contactor coil wiring
9. **CAN bus** — add inverter CAN, shunt CAN
10. **HV connections** — final step, after all 12V functionality is verified

Build incrementally and verify each step before adding the next. A fault in step 3 is easy to find. The same fault buried under 8 more steps takes hours.

The Safety Box Harness

The BMW Safety Box (S-BOX) is often used in BMW-based conversions to provide pack monitoring and HVIL in one unit. The S-BOX harness is covered in the wire harness series (May 2024, Parts 1 and 2). Key point: the S-BOX communicates over CAN2 and provides the `udc` reading that the VCU uses for precharge. Set `shuntType` = SBOX and `shuntCan` = CAN2.

Common Wiring Mistakes

Mistake	Consequence	Prevention
VCU on ignition-switched power	Shunt loses UDC, precharge fails every startup	Permanent power to Pin 56 always
Brake switch on normally-closed terminal	`din_brake` stuck on, no torque ever	Use normally-open (NO) terminal
CAN H and CAN L swapped	No communication, no fault message	Use twisted pair, check colour code
Missing CAN termination	Intermittent communication faults under load	120Ω at each end of each bus
throtmax left at 0	Throttle does nothing	Set to 100 — see C04
Contactor coil+ wired to VCU output pin	VCU output damaged — outputs are low-side only	Coil+ to 12V, coil– to VCU pin

Source: Good Enuff Garage "DIY Wire Harness" series (April–June 2024, 10 videos) · Damien Maguire @Evbmw

Last verified against firmware: V2.30A (August 2025)

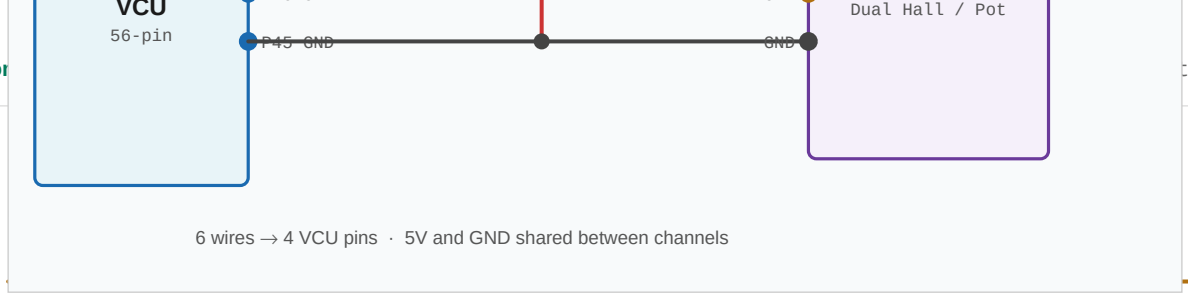


Figure: Throttle & Brake Wiring — wiring overview

Overview

The throttle and brake inputs are the primary driver interface signals. Getting them right — especially the throttle calibration — is essential for safe, predictable vehicle behaviour. This module covers the wiring; calibration parameters are covered in Module C04.

Throttle Pedal Wiring

Dual vs Single Channel

The ZombieVerter supports both single and dual-channel throttle pedals. **Dual-channel is strongly recommended** for safety — if one channel fails (open circuit, short circuit, or mechanical linkage failure), the VCU can detect the discrepancy between channels and cut torque rather than applying full throttle.

Single-channel is available but provides no redundancy. If the single pot fails in the high direction, the result is unintended full throttle. Use dual-channel wherever possible.

Pin Assignments

VCU Pin	Function	Connect to
Pin 48	Throttle 5V supply	Throttle pot(s) positive (Vcc)
Pin 47	Throttle channel 1	Channel 1 wiper
Pin 46	Throttle channel 2	Channel 2 wiper
Pin 45	Throttle ground	Throttle pot(s) negative (GND)

For a dual-channel pedal, both pots share the 5V supply (Pin 48) and ground (Pin 45), with their wipers going to Pins 47 and 46 respectively.

For a single-channel pedal, only Pins 48, 47, and 45 are used. Set `potmode` to single in the web interface.

Wiring Diagram

```
Pin 48 (5V) ■■■■■■■■■■■■■■■■■■■■ Pot 1 Vcc
■■■■■■■■■■■■■■ Pot 2 Vcc

Pin 47 (Ch1) ■■■■■■■■■■■■■■■■■ Pot 1 Wiper
Pin 46 (Ch2) ■■■■■■■■■■■■■■■■■ Pot 2 Wiper

Pin 45 (GND) ■■■■■■■■■■■■■■■■■■■■ Pot 1 GND
■■■■■■■■■■■■■■ Pot 2 GND
```

```
Pin 48 (5V) ■■■■■■■■■■■■■■■■■ Pot Vcc
Pin 47 (Ch1) ■■■■■■■■■■■■■■■■■ Pot Wiper
Pin 45 (GND) ■■■■■■■■■■■■■■■■■ Pot GND
```

Understanding How the ZombieVerter Reads Throttle

This is one of the most important things to understand before calibrating — **the ZombieVerter handles potmin/potmax differently from most inverters and motor controllers.**

The Standard Approach (Most Controllers)

Most motor controllers set `potmin` and `potmax` *outside* the mechanical travel range of the pedal. For example, if the pedal physically travels from an ADC value of 400 to 2600, you might set `potmin` to 300 and `potmax` to 2700. This way, if a value outside those limits is detected (due to an open wire, short, or mechanical failure), the controller faults safely.

The ZombieVerter Approach (Opposite!)

The ZombieVerter sets `potmin` and `potmax` *inside* the mechanical travel range:

- `potmin` should be set **just above** the value seen at the resting (off-throttle) position
- `potmax` should be set **just below** the value seen at full throttle

This means the pedal physically travels slightly beyond both limits. The VCU uses these values to calculate a normalised 0–100 `potnom` value.

> **This trips up almost everyone coming from other systems.** If you set `potmin` below your actual resting position, the VCU will interpret the pedal as always slightly depressed. If you set `potmax` above your actual WOT value, you'll never reach 100% torque.

1. Watch the `pot` spot value in the web interface while moving the pedal
2. Note the value at rest — set `potmin` just above this
3. Note the value at full travel — set `potmax` just below this
4. Do the same for `pot2min` / `pot2max` if using dual-channel

Throttle Parameters Quick Reference

Parameter	Description
`potmin`	ADC value just above off-throttle resting position
`potmax`	ADC value just below wide-open throttle
`pot2min`	Same as potmin but for channel 2
`pot2max`	Same as potmax but for channel 2
`potmode`	`single` or `dual` — must match your wiring
`potnom`	Resulting normalised 0–100 throttle value (spot value, read-only)
`throtmin`	Minimum allowed potnom — can be negative (for throttle-lift regen)
`throtmax`	Maximum allowed potnom in forward direction
`throtramp`	Rate of potnom increase (potnom change per %/10ms) — limits rate of torque rise
`throtramprpm`	Motor RPM above which throtramp no longer applies
`revlim`	Rev limiter (motor RPM)

Critical Gotcha: `throtmax` Defaults to Zero

`throtmax` defaults to **0** in a fresh configuration. This means the VCU will allow zero forward torque regardless of throttle position. **This is the most common reason for a "throttle does nothing" problem on first start.** Set `throtmax` to 100 (or your desired maximum) before expecting torque output.

Brake Signal Wiring

Purpose

The brake signal serves two functions in the ZombieVerter:

1. **Safety interlock** — `din_break` must read "off" before the VCU will allow torque output. If the brake is stuck "on", the VCU will not produce torque even if throttle is applied.
2. **Regen trigger** — applying the brake can trigger regenerative braking (configured separately — see Module A01).

Wiring

The brake input is a simple digital input:

VCU Pin	Function	Connect to
Pin 49	Brake input	12V+ when brake pedal is pressed

When the brake pedal is pressed, Pin 49 is pulled to 12V+. When released, Pin 49 should return to 0V (or float).

Do not leave Pin 49 floating — a floating input can read randomly as "on" due to electrical noise, blocking torque output unpredictably. If you are not wiring a brake signal for now (bench testing), either connect Pin 49 to ground through a resistor or confirm `din_break` reads "off" in spot values before troubleshooting torque issues.

BrakeLight Output

The VCU can drive a brake light output from a configurable low-side output pin. Assign the `BrakeLight` function in the web interface to activate this output when the brake is pressed above a configurable threshold.

Forward / Reverse Wiring

VCU Pin	Function	Connect to
Pin 54	Forward	12V+ when forward selected
Pin 53	Reverse	12V+ when reverse selected

Both are digital inputs pulled high to 12V+ to select direction. The direction signal can come from:

- A physical gear selector lever with detents (park/reverse/neutral/drive)
- A toggle switch
- A momentary button (set `dirmode` to `button` or `buttonReversed` in the web interface)

`dirmode` parameter options:

- `switch` — pin high = direction active
- `button` — momentary press toggles direction
- `switchReversed` — inverted logic
- `buttonReversed` — inverted momentary logic

Common Wiring Mistakes

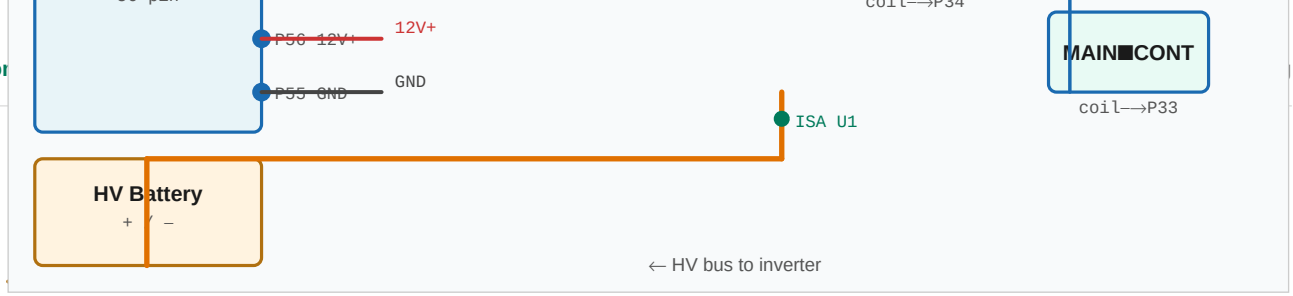
Problem	Cause
`din_break` always shows "on"	Brake pin wired to constant 12V+, or brake switch wired NC instead of NO
`din_break` always shows "on"	Floating pin picking up noise — add pull-down resistor
`potnom` never reaches 100	`potmax` set higher than actual WOT ADC value
`potnom` shows value at rest	`potmin` set lower than actual resting ADC value
No torque output	`throtmax` set to zero (default)
Erratic torque	Poor ground connection between throttle pot GND and Pin 45
Channel 2 fault	`potmode` set to dual but pot2min/pot2max not calibrated

What's Next

- **C04** — Throttle Calibration: step-by-step parameter tuning in the web interface
- **W03** — Contactor & Precharge Circuit
- **A01** — Regen Tuning: brake and throttle-lift regen configuration

Source: openinverter.org/wiki/ZombieVerter_VCU | openinverter.org forum threads

Last verified against firmware: V2.30A (August 2025)



Contactor & Precharge Circuit — wiring overview

Why This Module Matters

Contactor and precharge wiring is the most safety-critical part of a ZombieVerter installation. Getting it wrong can result in:

- Violent inrush current that welds contactors closed permanently
- Failure to enter run mode (the most common first-start problem)
- Unexpected HV live conditions on the vehicle chassis

Read this module completely before wiring. Do not skip the UDC section.

What the Contactor System Does

The ZombieVerter manages three contactors in a standard EV conversion:

Contactor	Purpose
Negative	Connects the battery negative to the HV system ground
Precharge	Connects HV positive through a resistor, to slowly charge inverter capacitors
Positive (Main)	Connects battery positive directly to the HV bus once precharge is complete

The sequencing logic is:

```
START signal received
■
▼
Negative contactor CLOSES ■■■■ always first
■
▼
Precharge contactor CLOSES ■■■■ current limited by precharge resistor
■
▼
UDC voltage rises toward pack voltage
■
▼
UDC ≥ udcsw setpoint? ■■■■ NO ■■■■ wait (up to 5 seconds)
■ ■
YES PRECHARGE FAULT
■
▼
Main (positive) contactor CLOSES
■
▼
VCU enters RUN mode ■■■■ opmode = "run", invstat = "on"
```

UDC — The Most Important Concept in This Module

UDC is the VCU's measurement of HV bus voltage. It is the single most critical input to the contactor sequencing logic.

Without a valid UDC reading, the ZombieVerter **will fail precharge and never enter run mode.** This is the most common cause of first-start failures.

The VCU can receive UDC from several sources, selected via the `shuntType` parameter:

shuntType value	Source
`ISA`	ISA IVT current/voltage shunt (most common)
`SBOX`	BMW S-BOX battery management unit
`VAG`	Volkswagen/Audi battery system
`LEAF`	Nissan Leaf inverter (reports pack voltage via CAN)

You must configure `shuntType` to match your actual hardware. If it is set to a source that is not present or not communicating, UDC will read zero and precharge will fail immediately.

ISA Shunt — Permanent Power Requirement

If you are using an ISA IVT shunt (the most common choice), it must have **permanent 12V power** — the same always-on supply as the VCU itself. The ISA shunt continuously monitors and reports UDC.

If the shunt is powered from an ignition-switched supply, it will lose its UDC reading when the key is off. When you next turn the key and start precharge, the VCU may see a stale or zero UDC value and behave unpredictably.

> **Rule:** ISA shunt power = always on. Same feed as the VCU.

Contactor Wiring

Electrical Concept: Low-Side Switching

All contactor outputs on the ZombieVerter are **low-side switches**. This means:

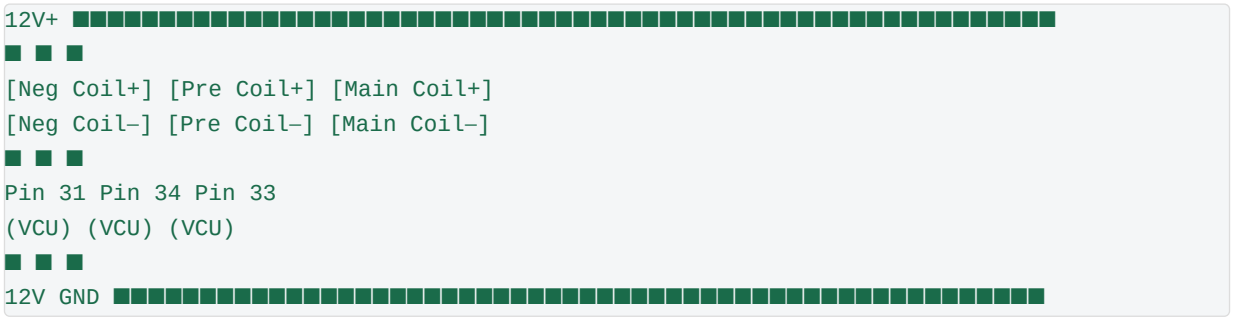
- The VCU output pin connects to the **ground leg** of the contactor coil
- When the VCU activates the output, it pulls that pin to ground, completing the circuit
- The **positive leg** of every contactor coil must be connected to 12V+

Never connect 12V+ to the VCU contactor output pins. These are ground-side switching outputs only.

Pin Assignments

VCU Pin	Function	Connects to
Pin 31	Negative contactor	Ground leg of negative contactor coil
Pin 33	Positive (main) contactor	Ground leg of positive contactor coil
Pin 34	Precharge contactor	Ground leg of precharge contactor coil

Wiring Diagram (Conceptual)



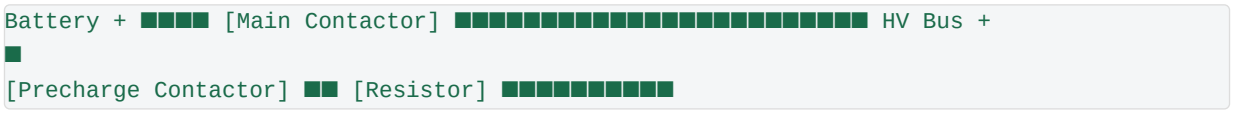
Contactor Polarity

Some contactors are polarity-sensitive — their coil has a defined positive and negative terminal. If wired backwards, they will not close (or may close but with reduced hold force and reduced life). Check your specific contactor datasheet.

Precharge Resistor

The precharge resistor limits current flow during the precharge phase, protecting both the contactor and the inverter capacitors from inrush damage.

- Typical values: 40Ω–100Ω at 50W–100W rating
- Higher resistance = slower precharge (more protection, but must complete within 5 seconds)
- Lower resistance = faster precharge (less protection, may stress contactors)
- 100Ω / 100W is a common and safe starting point for most conversions up to ~400V



UDC Measurement Point — ISA Shunt

The ISA shunt must be positioned to measure the voltage at the correct point. The shunt's U1 terminal reads voltage relative to HV ground.

During precharge, U1 should be connected to the HV bus on the inverter side of the main contactor. As the precharge resistor charges the inverter capacitors, U1 rises toward pack voltage. When it crosses `udcsw`, the VCU closes the main contactor.

A common mistake is connecting U1 to the battery side of the main contactor. In this position, U1 reads full pack voltage immediately (before precharge even starts), and the VCU will try to close the main contactor without actually precharging the inverter capacitors — potentially welding the main contactor on first use.

> **Check:** During initial power-up with no HV connected, U1 should read 0V. Apply HV with only precharge closed — U1 should rise slowly from 0V toward pack voltage. Only then will the VCU close the main contactor.

Key Parameters

These parameters control contactor and precharge behaviour:

Parameter	Description	Typical Value
`udcmin`	Minimum pack voltage — below this, VCU will not enter run mode	Set to ~80% of your minimum expected pack voltage (e.g. 20V for bench testing)
`udclim`	Maximum pack voltage — above this, VCU derates	Set just above your fully charged pack voltage
`udcsw`	Voltage threshold that triggers main contactor close	Set to ~95% of your nominal pack voltage
`shuntType`	Source of UDC measurement	Match to your hardware: ISA, SBOX, VAG, or LEAF

Common First-Start Mistakes

Symptom	Likely Cause
UDC reads 0, precharge never completes	`shuntType` wrong, shunt not powered, shunt not initialised, shunt CAN bus not connected
UDC reads full pack voltage immediately, main contactor fires instantly	ISA U1 connected to battery side instead of inverter side
Precharge times out (stays in PRECHARGE state)	`udcsw` set too high, precharge resistor too large, shunt not reporting correctly
VCU never attempts precharge	`udcmin` set too high (pack voltage is below the minimum threshold)
Precharge completes but VCU won't enter run mode	`din_break` is showing "on" — brake signal is active or stuck, blocking run mode

---

ISA Shunt Initialisation

A new ISA shunt must be initialised before it will communicate on CAN. This is a one-time procedure:

1. Wire the ISA shunt to permanent 12V and the correct CAN bus
2. In the web interface under **Comms**, set `shuntCan` to the CAN bus the shunt is connected to
3. Save parameters to flash
4. Under **Comms**, set `ISAMode` to **Init**
5. Save parameters to flash
6. Power cycle the VCU and shunt simultaneously (they should be on the same 12V feed)
7. The shunt initialises
8. Set `ISAMode` back to **Normal**
9. Save parameters to flash
10. Reboot the VCU

Separate the shunt and VCU power supplies. Power on the shunt first, wait 2–3 seconds, then power on the VCU. This delay allows the shunt to fully boot before the VCU attempts to communicate with it. This has resolved initialisation failures for a number of users.

Contactor Firing During Precharge — Inverter Enable Relay

One subtle but important wiring detail: the inverter (and charger, DCDC) should not receive its 12V "ignition/enable" signal until the VCU is ready to energise it. Premature inverter power-up can cause:

- CAN bus conflicts before the VCU has configured the inverter
- Inverter self-initialisation that conflicts with VCU sequencing

The ZombieVerter controls an external relay (via Pin 32, low-side switching) to provide a switched 12V "ignition" signal to these devices. This relay should only fire after the VCU is in a valid state.

```
12V+ ■■■■ [Relay coil +]
[Relay coil –] ■■■■ Pin 32 (VCU) ■■■■ GND
[Relay switch NO] ■■■■ 12V+
[Relay switch COM] ■■■■ Inverter/charger enable/ignition pin
```

Check that this relay is firing during precharge by watching the inverter's power indicator or CAN traffic when you apply the Start signal.

Troubleshooting Checklist

Before assuming a hardware fault, verify all of the following:

- [ ] `shuntType` matches actual hardware
- [ ] ISA shunt has permanent 12V power (not ignition-switched)
- [ ] ISA shunt has been initialised (ISAMode Init → Normal cycle)
- [ ] ISA shunt CAN bus matches `shuntCan` parameter setting
- [ ] CAN bus has correct termination (120Ω at each end of the bus)
- [ ] ISA shunt U1 is connected to inverter side of main contactor (not battery side)
- [ ] All contactor coil positive leads are connected to 12V+
- [ ] All contactor coil negative leads are connected to the correct VCU pins (31, 33, 34)
- [ ] `udcmin` is set to a sane value (try 20V for bench testing)
- [ ] `udcsw` is set below actual pack voltage
- [ ] `din_break` reads "off" in spot values (brake signal not stuck on)
- [ ] Inverter enable relay is firing during precharge (Pin 32)

What's Next

- **W04** — HVIL: High Voltage Interlock Loop
- **W07** — CAN Bus Wiring: termination and topology
- **C03** — Essential Parameters: First Start

Source: openinverter.org/wiki/ZombieVerter_VCU | openinverter.org forum threads

Last verified against firmware: V2.30A (August 2025)

TRACK WIRING · MODULE W04

HVIL — High Voltage Interlock Loop

What Is HVIL?

The High Voltage Interlock Loop (HVIL) is a low-current 12V detection circuit that passes through every HV connector, service disconnect, and access panel in the system. If any connector is unplugged, any panel is removed, or any HV cable is broken, the loop opens — and the VCU immediately opens the HV contactors.

HVIL transforms accidental HV contact from a likely fatal event into a contactor-open event. Instead of unplugging an HV cable while the system is live, you unplug the HVIL pin first (or simultaneously), the VCU sees the broken loop, and the HV bus drops before you can touch a live terminal.

How It Works

The HVIL circuit is a simple series loop:

```
VCU HVIL out → Connector 1 loop → Connector 2 loop → Battery disconnect → VCU HVIL in
```

Each HV connector in the system has two small signal pins (the HVIL pins) that are internally shorted together within the connector. When all connectors are fully mated, the loop is complete and current flows. When any connector is pulled, the loop opens, current stops, and the VCU opens the contactors.

The VCU monitors the loop return signal. A break in the loop triggers an immediate contactor open regardless of opmode.

The BMW Safety Box (S-BOX)

A commonly used solution in ZombieVerter BMW builds is the BMW Safety Box. The S-BOX is an OEM battery isolation module from early E-series BMW/Mini hybrid vehicles that combines:

- Negative and positive contactors with precharge
- HVIL monitoring
- Pack voltage monitoring (CAN)
- Manual service disconnect

For BMW-based conversions it provides all of these functions in a single tested unit. Set `shuntType` = SBOX in VCU parameters.

The S-BOX communicates over CAN2 by default. Set `shuntCan` = CAN2. The S-BOX provides `udc` to the VCU — this is what drives precharge completion detection.

Wiring HVIL From Scratch

For builds not using the S-BOX, HVIL can be implemented with:

S-BOX Integration

VCU Wiring

S-BOX terminal	Connect to
12V+	Permanent 12V (same feed as VCU)
`GND`	Chassis ground
CAN H	VCU CAN2H
CAN L	VCU CAN2L
Contactor coil– (neg)	VCU Pin 31
Contactor coil– (pos)	VCU Pin 33
Precharge coil–	VCU Pin 34
HV+	Battery positive (via fuse)
HV–	Battery negative

VCU Parameters

```
shuntType = SBOX
shuntCan = CAN2
```

The VCU will send CAN commands to the S-BOX to open and close its internal contactors as part of the normal precharge sequence.

Connectivity Check

With S-BOX powered and CAN connected:

- Check `udc` in Spot Values — should show a non-zero value if the S-BOX is communicating and the HV battery is connected
- Check `status` — should not show a CAN communication error

HVIL on OEM Drive Unit Connectors

Many OEM inverters and drive units have built-in HVIL connectors:

Without HVIL

Builds without HVIL are possible — many simple conversions omit it. Without HVIL:

- Disconnecting any HV connector while the system is live is potentially fatal
- The VCU has no automatic protection against accidental HV exposure during service
- You rely entirely on the manual service disconnect and your own procedural discipline

If omitting HVIL, the discharge procedure (Module F04) must be followed rigorously every single time before touching any HV component. There are no shortcuts.

Source: Good Enuff Garage BMW Safety Box series (April–May 2024) · openinverter.org/wiki

Last verified against firmware: V2.30A (August 2025)

TRACK WIRING · MODULE W05

Cooling System Control

Why Cooling Matters More in a Conversion

In an OEM EV, the cooling system is engineered alongside the drivetrain with precise flow rates, thermal mass, and thermostat control tuned for that specific motor and inverter. In a conversion, you are building a cooling system from scratch around salvaged components that were designed with different assumptions.

Under-cooling in a conversion manifests as:

- Thermal derating — the inverter reduces available torque as temperature rises
- Thermal shutdown — the inverter or motor stops completely to protect itself
- Accelerated component degradation over time

Over-cooling wastes energy running pumps and fans at full speed continuously. The VCU can manage both.

What the VCU Can Control

The VCU has dedicated output functions for cooling components, assignable via the Dout section of the web interface:

Function	What it controls	How it activates
`CoolantPump`	Electric coolant pump relay	On during pre-charge, charge and run mode
`GS450Hpump`	GS450h oil pump PWM	PWM speed control via PumpPWM parameter
`CoolingFan`	Cooling fan relay or speed controller	Activates when temp exceeds FanTemp
`BrakeVacPump`	Brake vacuum pump relay	Activates when brake vacuum is low

Coolant Pump

An electric coolant pump must run whenever the inverter is active — from precharge through to run mode shutdown. Assign `CoolantPump` to a low-side digital output pin.

Cooling Fan — FanTemp (V2.20A+)

`FanTemp` sets the inverter heatsink temperature (`temphs`) at which the VCU activates the cooling fan output.

```
FanTemp = 40 ← fan on when heatsink reaches 40°C
```

Set this based on your cooling system design. A well-designed cooling loop should keep the inverter heatsink below 60°C under sustained load. If `temphs` regularly reaches 80°C+, the cooling system needs more capacity.

Brake Vacuum Pump

Many non-hybrid donor vehicles use engine vacuum for brake servo assistance. In an EV conversion with no engine, this vacuum disappears. An electric vacuum pump is required for brake servo operation.

The VCU can monitor a vacuum sensor and activate a vacuum pump as needed:

1. Assign `BrakeVacSensor` to an analogue input pin
2. Assign `BrakeVacPump` to a digital output pin
3. The VCU activates the pump when vacuum drops below threshold

Alternatively, use a standalone electric vacuum pump controller — these are available as off-the-shelf units that self-regulate without VCU integration.

Temperature Monitoring

The VCU receives temperature data from several sources:

Spot value	Source
`temphs`	Inverter heatsink temperature (from inverter CAN)
`tempm`	Motor temperature (from inverter CAN, where supported)
`tempcool`	Coolant temperature (if thermistor connected to VCU analogue input)

For vehicle integration, `temphs` and `tempm` can be used to drive the OEM temperature gauge (where the vehicle class supports it) — giving the driver a real indication of drivetrain thermal state.

Thermal Derating

Most supported inverters implement their own thermal derating — as heatsink temperature rises above a threshold, the inverter reduces available torque. This is done inside the inverter firmware, not by the VCU. The VCU sees reduced torque output but cannot override inverter-internal derating.

Signs of thermal derating:

- Throttle at 100% but `torque` spot value below maximum
- `temphs` above 60–70°C
- Performance gradually reduces after sustained high-load driving

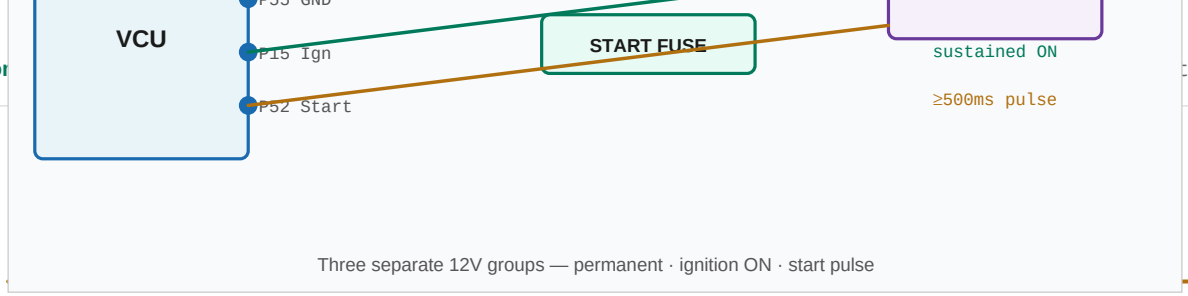
Solution: more cooling. Larger radiator, higher flow pump, or additional cooling surface.

Common Cooling Mistakes

Source: Damien Maguire @Evbmw — V2.20A firmware walkthrough (December 2024) · openinverter.org community builds

PumpPWM and FanTemp added V2.20A

Last verified against firmware: V2.30A (August 2025)



Three separate 12V groups — permanent · ignition ON · start pulse

12V Auxiliary & Ignition Wiring — wiring overview

Overview

The ZombieVerter's 12V wiring is divided into three logical groups:

1. **Permanent power** — always on, powers the VCU itself
2. **Ignition ON** — switched when the key is turned to "on" position
3. **Start** — momentary signal when the key is turned to "start" position

Understanding why these are separate — and keeping them separate — is important for correct contactor sequencing and reliable operation.

Permanent Power

The VCU must remain powered at all times, even when the vehicle is "off." This is required for:

- Charge timers
- Charging session management
- Monitoring systems during parking
- ISA shunt continuous UDC reporting

VCU Pin	Function
Pin 56	12V+ permanent positive
Pin 55	12V– ground

- Bench power supply (recommended — set to 13V, limit to 1–2A)
- Car battery (add a fuse)
- 9V battery (works for basic power-up verification only — not for any functional testing)

Do not use an ignition-switched supply for permanent power. The VCU and ISA shunt must stay on when the key is off. See Module W03 for why this matters for precharge.

Fuse Block Strategy

For a clean installation, use separate fuse blocks for:

Fuse block	Powers	Notes
Permanent power block	VCU (Pin 56), ISA shunt, any always-on devices	Direct from battery through main fuse
Ignition ON block	Devices that should power on with key	Fed from VCU-controlled relay (Pin 32 output)
Start position block	Start signal only	Momentary feed

Using separate fuse blocks makes troubleshooting far easier — you can isolate each logical group.

Ignition ON Signal

When the driver turns the key to the "on" position (not start), Pin 15 must see 12V.

VCU Pin	Function	Signal type
Pin 15	Ignition ON	12V switched — stays on while key is in ON position

This is not a momentary signal. It stays high for the entire time the vehicle is in use.

Start Signal

When the driver turns the key to the "start" position (momentary crank position), Pin 52 must see a brief 12V pulse.

VCU Pin	Function	Signal type
Pin 52	Start	Momentary 12V — brief pulse, like turning a key to start

The start signal triggers the precharge sequence. It does not need to stay high — a pulse of at least ~500ms is sufficient. The VCU latches into precharge mode once triggered.

The Inverter/Charger Enable Relay (Pin 32)

The VCU controls an external relay through Pin 32 (low-side switching) to provide a managed "ignition" or "enable" signal to the inverter, charger, and DC-DC converter. These devices should not receive their enable signal until the VCU is ready for them.

- Pin 32 is a low-side output — it pulls the relay coil ground to ground when activated
- The relay switches 12V to the enable pins of the inverter, charger, and/or DC-DC
- The VCU controls when this relay fires based on operating state

```
12V+ ■■■■■■■■■■■■ [Relay coil +]
[Relay coil −] ■■■■■ Pin 32 (VCU) ■■■■■ GND

12V+ ■■■■■■■■■■■■ [Relay switch COM]
[Relay switch NO] ■■■■■ Inverter enable / ignition input
■■■■ Charger enable input
■■■■ DC-DC enable input
```

This relay is sometimes called the "inverter relay" or "ignition relay" in forum discussions. It's the same thing — a VCU-controlled switch that powers up the drivetrain components when the VCU is ready.

Forward and Reverse Signals

VCU Pin	Function	Signal type
Pin 54	Forward	12V when forward direction selected
Pin 53	Reverse	12V when reverse direction selected

Direction Mode Options

Configure via `dirmode` parameter (under Throttle in the web interface):

dirmode setting	How it works
`defaultFwd`	No wiring needed for forward. Default direction is forward when in run mode. Add a button on Pin 53 to select reverse.
`button`	Use a momentary button. No pin active = default direction. Pin active = alternate direction.
`switch`	Use a 3-position switch. Pin 54 high = forward, Pin 53 high = reverse, neither high = neutral.
`switchReversed`	Same as switch but motor direction inverted. Use if drivetrain spins backwards.
`buttonReversed`	Same as button but inverted.

Behaviour in Each Mode

- Position 1 → 12V to Pin 54 → Forward/Drive
- Position 2 → Centre (neither pin active) → Neutral
- Position 3 → 12V to Pin 53 → Reverse

The direction shown in spot values will track the switch position. Note: the switch wiring can be counterintuitive — the "Forward" label on the switch often corresponds to the opposite blade terminal. Check your specific switch's wiring diagram.

Direction goes to Neutral when opmode goes to Off. When you next start the vehicle with the switch in Forward, it will immediately go to Drive — correct behaviour.

Torque Direction

Forward sends positive torque commands to the inverter. Reverse sends negative torque commands — the motor spins in the opposite direction. There is no physical reverse gear in most EV transaxles (Prius, Outlander, Leaf) — reverse is achieved by commanding negative torque.

Brake Signal

VCU Pin	Function	Signal type
Pin 49	Brake	12V when brake pedal pressed

The brake signal is a safety interlock. `din_break` must read "off" (brake not pressed) for the VCU to allow torque output.

Wire to the normally-open (NO) terminal of the brake light switch, so Pin 49 sees 12V only when the brake pedal is depressed.

The "Holy Trinity" — Minimum for Run Mode

From bench testing experience, the minimum wires required to reach run mode are:

1. **Pin 56** — permanent 12V+

2. **Pin 55** — permanent ground
3. **Pin 15** — ignition ON (12V switched)
4. **Pin 52** — start (momentary 12V)
5. **Pin 54 or 53** — forward or reverse direction

Plus correct parameters (especially `udcsw`, `shuntType`, and `throtmax`).

What's Next

- **W07** — CAN Bus Wiring
- **W03** — Contactor & Precharge Circuit
- **C03** — Essential Parameters: First Start

Source: "EV Conversion DIY Wire Harness" series, "ZombieVerter VCU Forward & Reverse" — Good Enuff Garage (Apr–May 2024)

Last verified against firmware: V2.30A (August 2025)

TRACK CONFIGURATION · MODULE C00

Firmware Version History

Why This Module Exists

The ZombieVerter firmware has evolved significantly since first release. YouTube videos, forum posts, and even parts of this training series may reference older behaviour, renamed parameters, or features that didn't yet exist at the time of recording. This module is the **ground truth reference** for what changed when.

- Before following instructions from any video or forum post, check the video date against this changelog
- If a parameter name doesn't match what you see in your firmware, look here for renames
- If a feature described in a module isn't visible in your web interface, check the version it was introduced

Firmware Version Summary Table

Version	Release date	Era	Key theme
1.00.A	Jun 2022	V1	First official release
1.06A	Nov 2022	V1	Throttle mode bug fix, throtdead introduced
1.07A	Nov 2022	V1	Leaf inverter bug fix
1.10A	Nov 2022	V1	CAN3, Leaf CAN improvements
1.11A	Feb 2023	V1	DC current limit fix
2.00A	**May 2023**	**V2 — major**	**IO matrix, virtual functions, module decoupling**
2.01A	Sep 2023	V2	Outlander charger, VW EBox shunt, BMS module
2.04A	Jan 2024	V2	IOMatrix analog inputs, gear shifter class, Tesla DCDC, OBD2
2.05A	Mar 2024	V2	Throttle UDC limit fix, LIN heater, Park gear, 250ms contactor delay
2.15A	Apr 2024	V2	Regen, Outlander rear inverter, GS450h auto shifting, brake light regen
2.17A	May 2024	V2	Outlander rear motor bug fix
2.20A	Dec 2024	V2	Foccci integration, PWM functions, E90 CAN, revRegen, Kangoo BMS
2.22A	Mar 2025	V2	IS300h fix, LIN fix, Leaf Gen3 PDM improvement

Version	Release date	Era	Key theme
2.30A	**Aug 2025**	**V2 — current**	**Throttle smoothing, derate simplification, Leaf charging fix**

The V1 → V2 Boundary (May 2023) — The Most Important Break

V2.00A was a major architectural rewrite. If you see content from before mid-2023, treat it with caution. Key changes:

- **IO Matrix introduced** — configurable input/output pin assignments via web interface. Before V2, many pins had fixed functions. The `Din`/`Dout` parameter sections in the web UI are a V2 feature.
- **Module decoupling** — Leaf stack components (inverter, PDM, charger) can now be used independently. Before V2, Leaf stack required all components together.
- **Charge interface decoupling** — Any OBC can now work with any charge interface (LIM, CHAdeMO). Previously coupled.
- **Subaru vehicle module added**
- **IMPORTANT:** Upgrading from V1.xx to V2.xx requires a power cycle after update (different RTC settings)

Detailed Release Notes

2.30A — August 2025 (Current)

■ **Current — this training series targets this version**

- Leaf reverse torque limiter removed
- Leaf charging fix for low current PP detection
- Throttle smoothing — `potnom` float fix
- Derate logic simplified — ramps removed
- Throttle smoothing — throttle ramping after derating added
- Tested in 3 vehicles before release: BMW E39 (GS450h/Tesla PCS/CHAdeMO), BMW E46 (Leaf Gen1/Outlander charger/i3 LIM CCS/Ampera heater), BMW E31 (Tesla LDU/Tesla Gen2 charger/Tesla DCDC/VW coolant heater/CHAdeMO)

2.22A — March 2025

- Lexus IS300h inverter fix
- LIN bus fix for VW coolant heater
- Nissan Leaf Gen3 PDM operation improvement

2.20A — December 2024

New parameters introduced (relevant to training modules):

New parameter	Function	Module
`revRegen`	Toggle regen on/off in reverse	A01
`PumpPWM`	Output pin function now selectable (was GS450h only)	H01, H05
`FanTemp`	Inverter or charger temp threshold for cooling fan	W05

New parameter	Function	Module
`TachoPPR`	Pulses per revolution for tachometer output	H05
`TorqueDerate`	Spot value — set any time potnom derates	C06

Other notable changes:

- Foccci one-click setup and ZombieVerter integration
- CpSpoof PWM output for CP signal spoofing
- GS450h oil pump PWM now a selectable output function (PumpPWM)
- Outlander charge CP spoofing no longer hard-coded
- E65 shifting now uses shifter class (no longer hard-coded)
- BMW E90 lock state over CAN, warning lights, vehicle speed
- Classic Vehicle option added — has tacho output, uses 12V inputs
- Speedo output option on GS450h oil pump pin
- Voltage and current limiting now actually tapering (was broken before)
- Kangoo BMS integration added
- Heater fix — checks for HeatReq assignment
- Outlander heater + heartbeat message
- GS450h/IS300h: option for blending MG1/MG2 until 50% potnom

2.17A — May 2024

- Fix Outlander rear motor bug (minor point release)

2.15A — April 2024

Regen introduced for the first time as a working feature.

Key additions:

- **Regenerative braking** — working implementation
- Outlander rear inverter support added
- Prius Gen3 temperature reading fix (was causing uneven throttle)
- Brake light output option triggered by regen
- **GS450h auto shifting** introduced
- JLR gear shifter support added
- No Vehicle CAN option added
- BMW CAN sleep and wake
- Throttle ramping fixed

2.05A — March 2024

- Throttle UDC limit bug fixed (was broken)
- Brake vacuum pump control fix (now turns off when ignition off)
- 250ms delay between HV contactor activations (needed for parallel packs)
- LIN-controlled VW heater added — **hardware note: R50 must be changed to 1kΩ and C45 to 1nF on boards ordered before 14/03/24**
- Park gear added to gear selection
- GS300h battery discharge limit fixed

2.04A — January 2024

Major feature additions:

- IO Matrix extended to analog inputs — brake vacuum pump control via vacuum sensor, charge port PP detection
- Gear shifter class added — supports 12V inputs and BMW F30 CAN-based shifter
- Tesla Gen2 DCDC converter class added
- OBD2 class for Torque Pro via CAN/ELM327
- BMW E31 vehicle class added
- OpenInverter CAN class updated

2.01A — September 2023

- Mitsubishi Outlander PHEV charger/DCDC support added
- VW PHEV EBox shunt option added
- BMS module merged (bench tested only at release)
- Heater bug fix

2.00A — May 2023 — MAJOR VERSION

See "V1 → V2 Boundary" section above for full impact.

1.06A — November 2022 — Important V1 note

> "Please check your potmin, pot2min, potmax, pot2max values: They are the ABSOLUTE minimum and maximum. Please set them in a way that the potval can never reach them."

This confirms the inside-the-range calibration method has been present since at least V1.06A.

New in V1.06A:

- `throtdead` parameter introduced (throttle deadzone)
- Dual-channel throttle limp mode: if channels deviate >10%, throttle limited to 50% and THROTTLE12DIFF error stored
- Regen disabled entirely (re-enabled properly in V2.15A)

Content Staleness Reference

Use this table when consuming training content from external sources (YouTube, forum posts):

Content date	Firmware era	Trust level	Notes
Before Jun 2022	Pre-release	■ **Outdated**	Pre-release firmware, significant changes since
Jun 2022 – Apr 2023	V1.00–V1.11	■ **Outdated**	V1 era — IO Matrix, regen, many features missing
May 2023 – Dec 2023	V2.00–V2.01	■ **Verify**	V2 architecture but many features not yet present
Jan 2024 – Mar 2024	V2.04–V2.05	■ **Verify**	Most wiring/config content valid, regen not working
Apr 2024 – Nov 2024	V2.15–V2.17	■ **Mostly current**	Regen works, most features present, minor gaps
Dec 2024 onwards	V2.20+	■ **Current**	Full feature set, matches this training series

Parameter Renames / Removals Between Versions

Parameter	Status	Notes
`ISAMode`	■ Stable	Present since V1, still current
`udcsw`	■ Stable	Present since V1, still current
`throtdead`	■ Stable	Introduced V1.06A
`revRegen`	■ Added V2.20A	Not present in older firmware
`PumpPWM`	■ Added V2.20A	Was GS450h-only before this
`FanTemp`	■ Added V2.20A	
`regenRamp`	■ Check	Listed as "new but not used yet" in V1.06A release notes — verify current status
`HVRequest`	■ Non-functioning	Listed in IO wiki as "NOT FUNCTIONING" — do not rely on
Outlander (inverter=6)	■ Deprecated	Replaced by RearOutlander (inverter=8)

The "Inverter Sequence Numbering" Warning

This affects anyone upgrading from V2.05A or earlier to V2.15A or later, and from any version to V2.22A.

The numeric values of the `inverter` parameter enum changed between versions. If you upgraded without reselecting your inverter type, the VCU may be trying to control the wrong inverter protocol.

1. Go to Parameters → Comms
2. Reselect your inverter type from the dropdown
3. Save to flash
4. Reboot the VCU

Staying Current

- **GitHub releases:** github.com/damienmaguire/Stm32-vcu/releases — always check here before updating
- **Forum thread:** openinverter.org/forum — development thread has release announcements
- **Damien's YouTube:** New firmware releases are typically accompanied by a walkthrough video

Source: github.com/damienmaguire/Stm32-vcu/releases — full release history

This module should be updated with each new firmware release.

Current latest firmware: V2.30A (August 2025)

TRACK CONFIGURATION · MODULE C01

Flashing Firmware

Do You Need This Module?

Probably not. Boards purchased from the EVBMW webshop come pre-programmed and ready to use. You do not need to flash firmware before first use on a new board.

You need this module if:

- You have a blank/unprogrammed board
- The web interface shows `firmware: null`
- A firmware update failed partway through and the board is unresponsive
- You want to run a specific firmware version

Know Your Chip

Before doing anything with firmware, identify which microcontroller is on your board.

Chip	Markings	Notes
STM32F107	ST logo + STM32F107	Standard chip — use main firmware branch
GD32F107	GD logo + GD32F107	Pandemic-era substitute — requires separate firmware branch

The GD32F107 ("GD chip") was used on some boards produced during the 2021–2022 semiconductor shortage. A separate firmware branch (`GD_Zombie`) exists on GitHub but development was abandoned shortly after release. As of the current main firmware (V2.20A), the GD branch has substantially diverged and is not kept up to date. **If you have a GD board, you are limited to an older firmware version and will miss recent features.**

If you are unsure which chip you have, look at the main IC on the board under good lighting. The manufacturer logo and chip number are printed on the chip.

Normal Firmware Update (Web Interface)

For routine firmware updates on a working board, use the web interface. No external tools required.

1. Download the latest firmware `.bin` file from the GitHub releases page
2. Connect to the VCU via Wi-Fi
3. Navigate to `192.168.4.1` in your browser
4. Find the **UART Update** field in the interface
5. Select your `.bin` file
6. Upload and wait for the update to complete

- The file must be named `stm32_vcu.bin` — rename it if it has a version suffix
- Windows 10 has known reliability issues with the UART update method. If you are on Windows 10, consider using Ubuntu (WSL2 is fine) or switching to the ST-Link method if the UART update fails
- Do not power-cycle the board mid-update

After reboot, connect to the web interface and check the firmware version displayed at the top of the page. If it shows `firmware: null`, the update failed and you will need to use the ST-Link recovery method below.

Full Recovery via ST-Link (Bricked Board)

Use this method if:

- The web interface shows `firmware: null`
- The board is completely unresponsive (no Wi-Fi, `acty` LED not flashing)
- UART update failed and board will not boot

What You Need

- ST-Link V2 dongle (cheap clones from Amazon/AliExpress work, though results vary)
- STM32CubeProgrammer software (free, from ST Microelectronics)
- Bootloader `.hex` file (from Johannes Huebner's bootloader repo)
- Latest ZombieVerter firmware `.hex` file (from Damien's GitHub releases)
- 12V power supply for the VCU

Use `.hex` files for ST-Link flashing (not `.bin`). The `.hex` format includes address information so STM32CubeProgrammer places the data correctly without you needing to manually specify flash addresses.

Procedure

1. **Download STM32CubeProgrammer** from the ST website and install it
2. **Upgrade your ST-Link firmware** using STM32CubeProgrammer — not strictly required but recommended
3. **Disconnect the Wi-Fi module** from the VCU board (gently pull the ESP8266 module from its socket) — this eliminates one potential source of interference during flashing
4. **Connect ST-Link to the VCU SWD test points:**

```
ST-Link pin → VCU test point
SWCLK → C
GND → G
SWDIO → D
```

5. **Apply 12V power** to the VCU (Pins 55 and 56) — the ST-Link alone does not power the board
6. **Open STM32CubeProgrammer** and connect to the device via the ST-Link interface — you should see the chip detected
7. **Perform a full chip erase** — this wipes all existing flash content including any corrupted firmware
8. **Flash the bootloader** `.hex` file first
9. **Flash the firmware** `.hex` file second
10. **Disconnect the ST-Link**
11. **Reconnect the Wi-Fi module**
12. Power cycle the VCU and verify the Wi-Fi access point appears and the web interface loads

If STM32CubeProgrammer Cannot Connect

- Check your SWD wiring — SWCLK and SWDIO are often swapped
- Try a different ST-Link clone — quality varies widely
- Ensure 12V is present on the VCU before attempting to connect
- Try with the Wi-Fi module disconnected if you haven't already

---

Serial Connection Parameters

If you use a serial terminal to communicate with the VCU (for diagnostics or the serial parameter interface):

```
Baud rate: 115200
Data bits: 8
Parity: None
Stop bits: 2 ← This is unusual! Most tools default to 1 stop bit.
Flow ctrl: None
```

The 2 stop bits is a common gotcha — if your serial terminal is not receiving coherent data, check the stop bit setting first.

Firmware Sources

What	Where
Latest stable firmware (`.bin` and `.hex`)	github.com/damienmaguire/Stm32-vcu/releases
Bootloader (`.hex`)	github.com/jsphuebner/tumanako-inverter-fw-bootloader/releases
GD32F107 firmware (older, unsupported)	github.com/damienmaguire/Stm32-vcu/tree/GD_Zombie
`STM32CubeProgrammer`	st.com/en/development-tools/stm32cubeprog.html

Always use a released version unless you have a specific reason to build from source. The GitHub releases page lists changelogs so you can see what changed between versions.

Building From Source (Advanced)

If you want to compile your own firmware from the repository:

```
git clone --recurse-submodules git@github.com:damienmaguire/Stm32-vcu.git
```

The `--recurse-submodules` flag is critical — it pulls in `libopeninv` and other dependencies. Without it, the build will fail with missing includes.

GD32F107 boards require specific code modifications — see the forum thread linked in the wiki before attempting to compile for a GD board.

What's Next

- **C02** — The Web Interface Walkthrough: navigating the UI once the board is running
- **C03** — Essential Parameters: First Start

Source: openinverter.org/wiki/ZombieVerter_VCU | openinverter.org/wiki/Zombieverter_programing

Last verified against firmware: V2.30A (August 2025)

TRACK CONFIGURATION · MODULE C02

Web Interface Walkthrough

Overview

All ZombieVerter configuration is done through a browser-based web interface served by the on-board ESP8266 Wi-Fi module. There is no desktop software to install, no USB cable required for normal operation, and no command line interaction needed for day-to-day use.

Connecting to the Web Interface

Minimum Requirements for First Power-Up

- A fully built ZombieVerter VCU
- Two wires connecting Pin 55 (ground) and Pin 56 (12V+) to a stable 12V supply
- A computer, tablet, or phone with Wi-Fi

Step-by-Step

1. Apply stable 12V power to Pins 55 and 56
2. The `acty` LED on the board should begin **flashing** — this confirms the MCU is running
3. On your device, scan for Wi-Fi networks and connect to the ZombieVerter access point:
- **SSID:** `inverter` or `zom_vcu` (varies by firmware version)
- **Password:** `inverter123`
4. Open a browser and navigate to: `192.168.4.1`
5. The openinverter web interface should load

If You Cannot Connect

Symptom: Wi-Fi network appears but browser cannot reach 192.168.4.1

Some recent boards use a new Wi-Fi module variant that does not automatically assign your device an IP address via DHCP. If this happens:

- Manually set your device's IP address to `192.168.4.2` (or any address in the `192.168.4.x` range other than `.4.1`)
- Set the subnet mask to `255.255.255.0`
- Set the gateway to `192.168.4.1`
- Try the browser again

Symptom: `acty` LED is not flashing

The MCU is not running. Check:

- 12V is present on Pin 56
- Ground is present on Pin 55
- If the board previously had a failed firmware update, it may need ST-Link recovery (see Module C01)

Interface Layout

The web interface is divided into several sections. The exact layout varies slightly between firmware versions, but the core structure is consistent.

Top Bar

Displays:

- Firmware version (if this shows `null`, the firmware is corrupted — see Module C01)
- Current VCU status / opmode
- Live spot values for key parameters (UDC, motor RPM, etc.)

Parameters Tab

The main configuration area. Parameters are grouped into categories:

Category	What's in it
Motor	Throttle limits, ramp rates, rev limiter, direction mode
Charger	Charger type, charge current limits, charge timer settings
Comms	CAN bus assignments, inverter type, shunt type, ISA mode, CANID settings
Throttle	potmin, potmax, pot2min, pot2max, potmode
Din	Configurable digital input pin assignments
Dout	Configurable digital output pin assignments
Temps	Temperature setpoints for fan, pump, heater enable
Contactors	udcmin, udclim, udcsw, shunt type
BMS	BMS current limits, cell voltage limits

Spot Values Tab

Live read-only values from the VCU. Essential for diagnostics and calibration. Key spot values:

Spot Value	What it shows
`pot`	Raw ADC reading from throttle channel 1
`pot2`	Raw ADC reading from throttle channel 2
`potnom`	Normalised throttle position (0–100)
`udc`	HV bus voltage from shunt
`invudc`	HV voltage reported by the inverter
`idc`	DC current from shunt
`opmode`	Current VCU operating mode
`invstat`	Inverter status
`din_brake`	Brake input state (must be "off" for torque)
`din_forward`	Forward input state
`din_reverse`	Reverse input state
`din_start`	Start input state
`speed`	Motor speed (RPM)

Spot Value	What it shows
`torque`	Requested torque
`temphs`	Heatsink temperature
`tempm`	Motor temperature
`PPVal`	Proximity pilot ADC value (if used)

Logging / Graphing

The interface includes a real-time graphing capability. Select which spot values to plot, set the update interval, and watch live traces. Useful for:

- Verifying throttle linearity
- Watching precharge voltage rise
- Diagnosing temperature issues
- Tuning regen behaviour

Save / Load

Parameters can be saved to flash (persistent across power cycles) or exported/imported as JSON files. Always **save to flash** after changing parameters — changes held only in RAM are lost on power cycle.

Parameter Workflow

The typical workflow for any configuration change:

1. Make the change in the Parameters tab
2. Observe the effect in Spot Values (some changes take effect immediately)
3. When satisfied, click **Save to Flash**
4. Reboot if required (some parameters only take effect after reboot)

Always save to flash before testing with HV. A power interruption during testing will otherwise revert your configuration.

Configuring Input/Output Pin Functions

Digital input and output pin assignments are made through the `Din` and `Dout` parameter categories. This is where you tell the VCU what function each configurable pin serves.

Inverter Type Selection

One of the first and most important settings is the `inverter` parameter under **Comms**. This tells the VCU which motor control protocol to use:

Value	Inverter
0	None — no inverter connected
1	Nissan Leaf Gen1/2/3 (CAN)
2	Lexus GS450h (sync serial)
3	UserCAN — not currently used

Value	Inverter
4	OpenInverter board (CAN)
5	Toyota Prius Gen3 (sync serial)
6	Outlander — deprecated
7	Lexus GS300h (sync serial)
8	Mitsubishi Outlander rear (CAN)

Set this before attempting to communicate with your inverter.

Vehicle Integration Type

The `vehicle` parameter selects the CAN profile for dashboard/instrument cluster integration:

Value	Vehicle
0	BMW E46
1	BMW E6x+ (E60, E90, etc.)
2	Classic — digital IO only, no vehicle CAN
3	None
5	BMW E39
6	VAG
7	Subaru
8	BMW E31

If you are not integrating with an OEM instrument cluster, set this to `None` or `Classic`.

ESP32 CAN Bus Web Interface (Alternative)

The VCU also supports an ESP32-based CAN bus web interface as an alternative to the built-in ESP8266. If you use this:

- The Node ID is **hard-coded to 3**
- Note that the FOCCCI (CCS charge controller) has a default Node ID of 22 — no conflict, but be aware if you are also using FOCCCI

What's Next

- **C03** — Essential Parameters: First Start
- **C04** — Throttle Calibration

Source: openinverter.org/wiki/ZombieVerter_VCU

Last verified against firmware: V2.30A (August 2025)

TRACK CONFIGURATION · MODULE C03

Essential Parameters: First Start

Overview

This module walks you through the minimum parameter set required to bring a ZombieVerter system to life for the first time. Follow these steps in order. Do not skip ahead to HV testing until the LV checks are complete.

Before You Start: LV Bench Test

You should be able to complete the entire first-start sequence with **12V only** — no HV battery connected. This lets you verify wiring, parameters, and sequencing logic before introducing any HV risk.

Required for LV bench test:

- ZombieVerter VCU with 12V power
- Wi-Fi connected device with web interface open
- Throttle pedal (or a potentiometer simulating one)
- Brake signal (a simple switch to 12V)
- Forward/Reverse signal (switches to 12V)
- Ignition ON and START signals (switches to 12V)
- Contactor coils connected (they will click — no HV needed to verify sequencing)
- ISA shunt connected and initialised (see Module W03)

Step 1: Set the Inverter Type

Set this to match your actual inverter. If you set it wrong, the VCU will send CAN messages on the wrong IDs at the wrong baud rate and nothing will communicate.

Your inverter	Set inverter to
Nissan Leaf (any gen)	1
Lexus GS450h	2
Toyota Prius Gen3	5
Mitsubishi Outlander (rear)	8
OpenInverter board	4
Nothing yet	0

Save to flash.

Step 2: Set the Shunt Type

Your UDC source	Set shuntType to
ISA IVT shunt	ISA
BMW S-BOX	SBOX

Your UDC source	Set shuntType to
VW/Audi system	VAG
Nissan Leaf inverter	LEAF

If you are using an ISA shunt and haven't initialised it yet, do that now — see Module W03. The VCU cannot precharge without a valid UDC reading.

Save to flash.

Step 3: Set the CAN Bus Assignments

The ZombieVerter has three CAN interfaces (CAN1, CAN2, CAN3). You need to tell the VCU which physical bus each device is connected to.

> **Note:** There is a known documentation issue where CAN1 and CAN2 are labelled in reverse in some older wiring diagrams. If you find your inverter is not communicating despite correct CAN wiring, try swapping the CAN bus assignment.

Common starting configuration:

- `inverterCan` = CAN1 (or whichever bus your inverter is on)
- `shuntCan` = same bus as the shunt wiring

Save to flash.

Step 4: Set Voltage Limits

Parameter	Description	Bench test value	Real vehicle value
`udcmin`	Minimum voltage — below this, VCU will not enter run	20V	~80% of minimum pack voltage
`udclim`	Maximum voltage — above this, VCU derates	High value (e.g. 500V)	Just above max charged pack voltage
`udcsw`	Voltage at which precharge is complete and main contactor closes	15V (for bench with no HV)	~95% of nominal pack voltage

For initial LV bench testing with no HV connected, set `udcsw` to a low value (15–20V) so precharge "completes" quickly and lets you verify the sequence.

Save to flash.

Step 5: Configure Throttle (Minimum Settings)

First, set `potmode`:

- `single` — one throttle pot
- `dual` — two throttle pots (recommended)

Then watch the `pot` spot value while moving the throttle pedal and set:

- `potmin` = value just **above** the resting (off-throttle) ADC reading
- `potmax` = value just **below** the full-travel ADC reading

Then set:

- `throtmin` = 0 (or a small negative value if you want throttle-lift regen)
- `throtmax` = 100

> `throtmax` **defaults to 0.** This is the single most common cause of "throttle does nothing." Set it to 100 before testing.

If using dual throttle, also calibrate `pot2min` and `pot2max` the same way while watching `pot2`.

Save to flash.

Step 6: Verify Brake Signal

In Spot Values, check `din_brake`.

- It should show **"off"** when the brake pedal is not pressed
- It should show **"on"** when the brake pedal is pressed

`din_brake` must be **"off"** for the VCU to allow torque output. If it is stuck "on" with no brake pressed, check:

- Brake switch wiring (is it normally open or normally closed?)
- Is the brake pin floating? Add a pull-down resistor to ground
- Is something else connected to Pin 49 at 12V?

Step 7: Verify Direction Inputs

Apply 12V to Pin 54 (forward) and check `din_forward` in Spot Values — should show "on".

Apply 12V to Pin 53 (reverse) and check `din_reverse` — should show "on".

Step 8: First Precharge Test (LV Only)

With everything configured and saved to flash:

1. Apply ignition ON signal to Pin 15 (pull to 12V)
2. Briefly apply START signal to Pin 52 (momentary 12V pulse)
3. Watch Spot Values:

- `udc` should update (even if reading 0V with no HV)
- Contactor outputs should fire (you'll hear the contactors click in sequence)
- If `udc` rises above `udcsw`, the main contactor should close
- `opmode` should eventually show `run`
- `invstat` should show `on`
- Was the START signal held long enough? It should be a short pulse, like turning a key — but it must be long enough for the VCU to register it (at least 500ms)
- Is `udc` reading a valid value? If it reads 0, the shunt is not communicating
- Is `udcsw` reachable? For LV bench testing, lower `udcsw` to 5V if needed
- Check `din_brake` is "off"
- Check `din_forward` or `din_reverse` is "on" (VCU needs a direction selected)
- Check `udc` is above `udcmin`

Step 9: First Torque Test (LV Only — No Motor Movement Expected)

With `opmode` showing `run`:

1. Apply forward direction (Pin 54 to 12V)
2. Slowly apply throttle
3. Watch `potnom` — it should rise from 0 to 100 as you press the pedal
4. Watch `torque` — it should rise proportionally

Even without HV or a connected inverter, the VCU should be sending torque commands via CAN. You can verify this with a CAN analyser if available.

Complete First-Start Parameter Checklist

Parameter	Set to	Notes
`inverter`	Match your hardware	Comms section
`shuntType`	Match your hardware	ISA, SBOX, VAG, or LEAF
`inverterCan`	CAN bus your inverter is on	
`shuntCan`	CAN bus your shunt is on	
`udcmin`	20V (bench) / ~80% min pack V	
`udclim`	500V (bench) / above max pack V	
`udcsw`	15V (bench) / ~95% nominal pack V	
`potmode`	single or dual	Match wiring
`potmin`	Just above off-throttle ADC reading	
`potmax`	Just below WOT ADC reading	
`pot2min`	Same for channel 2 (dual only)	
`pot2max`	Same for channel 2 (dual only)	
`throtmin`	0	
`throtmax`	**100**	Default is 0 — most common gotcha
`vehicle`	None or Classic	Unless integrating with OEM cluster

What's Next

- **C04** — Throttle Calibration: fine-tuning potmin/potmax, ramp rates, dual-pot plausibility
- **C05** — CAN Configuration: inverter CAN profiles, baud rates, message IDs
- **C06** — Fault Codes & Status Flags: reading and interpreting faults

Source: openinverter.org/wiki/ZombieVerter_VCU | openinverter.org forum threads

Last verified against firmware: V2.30A (August 2025)

TRACK CONFIGURATION · MODULE C04

Throttle Calibration

Throttle Pedal Types

Before calibrating, understand what kind of throttle pedal you have.

Hall Effect vs. Potentiometer

Modern drive-by-wire throttle pedals from roughly 2005 onwards use **Hall effect sensors** — magnetic, no physical contact between moving parts. Older pedals used **potentiometers** (physical wiper contact). The ZombieVerter parameter names still use "pot" as a historical holdover, but the calibration procedure is identical regardless of sensor type. Hall effect pedals are generally more reliable and longer-lasting.

Single vs. Dual Channel

All Hall effect pedals contain **two independent sensors** (two channels) for redundancy. The ZombieVerter supports both:

- **Dual channel (recommended):** Both sensors are wired and monitored. If one channel fails or reads an implausible value, the VCU can detect the discrepancy and cut torque safely.
- **Single channel:** Only one sensor wired. No redundancy — if the single channel fails high, the result is unintended full throttle. Use dual channel if at all possible.

Mounting Styles

Throttle pedals from donor vehicles come in different mounting configurations:

- **Floor-mounted / pedestal mount** — Common in European cars (BMW, Mercedes). The pedal mounts to the floor on a bracket.
- **Firewall-mounted** — Common in American and Japanese cars. Mounts directly to the firewall, hanging position.
- **Bracket-mounted** — Some pedals (like certain Chevy/GMC units) use a separate bracket that can be cut, heated, and welded for custom fitment. Useful for classic car conversions.

Popular Donor Pedals

Pedal	Notes
BMW E46 (floor mount)	All plastic, comes with separate bracket — buy the bracket too
Honda Fit	Metal construction — can be cut and welded. Popular choice.
Toyota Prius (Gen 3)	Hall effect, commonly available, pinout on openinverter wiki
Chevy Avalanche / Envoy	Metal body, firewall mount
Generic junkyard	Use Pick-n-Pull type yards; 50% off sales around holidays

Tip: Find a pedal with a known pinout diagram *before* buying. The openinverter wiki has pinout diagrams for BMW and Prius pedals. For others, search "[year] [model] throttle pedal pinout" or check alldata.com (subscription).

Wiring Six Wires to Four Pins

Every dual-channel Hall effect throttle pedal has **six wires** — three per channel:

- Ground (×2)
- 5V supply (×2)
- Signal/wiper (×2)

These six wires connect to **four VCU pins** by combining the shared grounds and shared 5V supplies:

```
Pedal wire → VCU pin
Channel 1 ground ■■■
Channel 2 ground ■■■■■ Pin 45 (throttle ground)

Channel 1 5V ■■■
Channel 2 5V ■■■■■ Pin 48 (throttle 5V supply)

Channel 1 signal ■■■■ Pin 47 (throttle channel 1)
Channel 2 signal ■■■■ Pin 46 (throttle channel 2)
```

Important: The throttle grounds and 5V supply connect to the VCU pins, not to vehicle chassis ground or the 12V battery. The VCU manages power to the throttle internally. Connecting throttle ground to chassis can cause instability.

BMW Pedal Color Guide (Version 1 — brown/yellow/white with green stripe variants)

Wire	Function
`Brown`	Channel 1 ground
Brown/green stripe	Channel 2 ground
`Yellow`	Channel 1 5V (or signal — verify with pinout)
Yellow/green stripe	Channel 2 5V (or signal)
`White`	Channel 1 signal
White/green stripe	Channel 2 signal

Always verify with the official pinout on the openinverter wiki before crimping.

Calibration Procedure

Step 1: Verify Wiring

With the VCU powered and connected to the web interface:

1. Navigate to **Spot Values**
2. Find `pot` and `pot2`
3. Move the throttle pedal slowly from rest to full travel
4. Both `pot` and `pot2` should **change** as you move the pedal

If `pot` or `pot2` does not change, the relevant channel is not wired correctly. Check your wiring before proceeding.

Step 2: Read the Resting Values

With the pedal **completely released** (foot off):

1. Read `pot` — note this value (e.g. 435)

2. Read `pot2` — note this value (e.g. 221)

Step 3: Read the Full-Travel Values

Press and **hold** the pedal at full travel:

1. Click Refresh in spot values
2. Read `pot` at full travel (e.g. 2410)
3. Read `pot2` at full travel (e.g. 1211)

Step 4: Set potmin and potmax

Navigate to **Parameters** → **Throttle**.

The ZombieVerter sets potmin/potmax **inside** the mechanical travel range (opposite to most inverters):

Parameter	Calculation	Example
`potmin`	Resting value **+ 10**	435 + 10 = **445**
`potmax`	Full-travel value – **10**	2410 – 10 = **2400**
`pot2min`	Channel 2 resting **+ 10**	221 + 10 = **231**
`pot2max`	Channel 2 full-travel – **10**	1211 – 10 = **1201**

Why ±10? This creates a small buffer inside the mechanical limits. The resulting `potnom` range (0–100) still covers 100% of usable throttle — you lose only a tiny fraction of ADC range in exchange for stability at the extremes.

Press **Enter** after each value, then **Save Parameters to Flash**.

Step 5: Set potmode

Your wiring	Set `potmode` to
Both channels wired	`dual`
One channel only	`single`

Step 6: Verify potnom

Go back to Spot Values. Watch `potnom`:

- At rest: should read **0** (or close to 0)
- At full travel: should read **100** (or close to 100)
- Movement should be smooth and proportional

If `potnom` reads a small non-zero value at rest, your `potmin` is set slightly too low — increase it by 5–10 counts.

Step 7: Set throtmin and throtmax

This is the most commonly missed step.

Parameter	Description	Typical value
`throtmin`	Minimum allowed potnom (can be negative for lift-off regen)	0 (or small negative for regen)
`throtmax`	Maximum allowed potnom in forward	**100**

`throtmax` **defaults to 0.** A VCU with `throtmax = 0` will never produce forward torque regardless of throttle position. If your throttle appears to do nothing, check this parameter first.

Additional Throttle Parameters

Parameter	Description
`throtramp`	Rate of potnom increase per 10ms interval. Limits how fast torque rises when pedal is pressed quickly. Higher = more responsive, lower = smoother.
`throtramprpm`	Motor RPM above which `throtramp` no longer applies. Above this speed, full throttle response is allowed.
`revlim`	Motor RPM hard rev limiter.
`throttledead`	Percentage of pedal travel that is deadband before torque begins.

Using the Spot Values Visibility Filter

The web interface spot values list is long. For throttle calibration, you only need to watch a few values. Use the **Visible** column to hide everything except:

- `pot`
- `pot2`
- `potnom`

Check the Visible box for those three, set the rest to non-visible, then use Auto Refresh. This keeps the display clean during calibration.

Dual-Channel Plausibility

With `potmode = dual`, the VCU continuously compares `pot` and `pot2`. If the two channels disagree by more than a configured threshold, the VCU will flag a throttle fault and cut torque.

This means **both channels must be calibrated correctly.** A common mistake is calibrating only channel 1 and leaving `pot2min`/`pot2max` at defaults — this will cause immediate plausibility faults when `potmode` is switched to dual.

Troubleshooting Throttle Issues

Symptom	Cause	Fix
`pot` or `pot2` doesn't change with pedal	Wrong pin, broken wire, or open circuit	Re-check wiring to pins 47, 46
`potnom` stuck at non-zero at rest	`potmin` set too low	Increase `potmin` by 5–10
`potnom` never reaches 100	`potmax` set too high	Decrease `potmax` by 5–10
No torque despite valid `potnom`	`throtmax` = 0	Set `throtmax` to 100
Throttle fault in dual mode	`pot2min`/`pot2max` not calibrated	Calibrate both channels
Erratic `potnom`	Poor ground between pedal and Pin 45	Check ground connection

What's Next

- **A01** — Regen Tuning: configuring `throtmin` negative values for lift-off regen
- **C03** — Essential Parameters: First Start (if not already completed)

Source: "ZombieVerter VCU Throttle Calibration" — Good Enuff Garage (May 2024) | openinverter.org/wiki/ZombieVerter_VCU

Last verified against firmware: V2.30A (August 2025)

TRACK CONFIGURATION · MODULE C06

Fault Codes & Status Flags

Overview

When the ZombieVerter doesn't behave as expected, the web interface gives you two key feedback mechanisms:

- `opmode` — the current operating state of the VCU
- `status` / `lastError` — what the VCU last tried to do and why it stopped

Understanding these is the difference between spending five minutes diagnosing a problem and spending five hours replacing things that weren't broken.

Operating Modes (`opmode`)

`opmode` is visible in Spot Values and shows the current state of the VCU state machine.

opmode value	Meaning
`off`	VCU is powered but ignition is not on, or system is in standby
`precharge`	Start signal received — negative and precharge contactors closed, waiting for UDC to rise
`pch` / `PCH`	Same as precharge (abbreviated display)
`run`	Precharge complete, main contactor closed, system ready for torque commands
`charge`	System is in charge mode (HV contactors closed for charging)

```
off → precharge → run
```

If the system goes `off → precharge → off` or `off → precharge → pch fail`, something prevented precharge from completing.

The `lastError` and `status` Fields

These two spot values tell you what the VCU was trying to do and why it stopped. They are often more useful than `opmode` alone.

> **Note on naming:** The field called `lastError` is not always actually an error — it often records the last significant status change. Think of it as "last known status" rather than "last fault." Similarly, `status` reports the current blocking condition.

Common `status` Messages

status message	Meaning	Common cause
`UDC below udcsw`	Pack voltage hasn't reached the precharge completion threshold	`udcsw` set too high, shunt not communicating, wrong `shuntType`

status message	Meaning	Common cause
`Throttle 1 2`	Throttle channel fault	`pot2min`/`pot2max` not calibrated, potmode mismatch, wiring issue
`din_break on`	Brake signal is active	Brake wired normally-closed instead of normally-open, floating input

The Most Common Failure: `udcsw`

This is the single most frequently missed parameter for first-time users.

Situation	Set `udcsw` to
Bench testing, no HV connected	0–5V (just to prove the sequencing works)
Bench testing with low-voltage HV source	Just below your source voltage
Real vehicle, known pack voltage	~95% of nominal pack voltage

Why does it default to 330V? Setting it too low is dangerous — the main contactor could close before the inverter capacitors are adequately charged, potentially damaging or destroying the inverter. The 330V default is conservative and safe for common Nissan Leaf/Prius systems. It is intentionally set to require you to make a conscious decision about your pack voltage.

1. Go to Spot Values, enable Auto Refresh
2. Turn ignition ON → press START
3. Watch `opmode` — does it go to `precharge`? Good. Does it then fail?
4. Check `status` — if it says `UDC below udcsw`, this is your issue
5. Go to Parameters → Contactor Control → `udcsw`
6. Set to 0 for bench testing (no HV) or appropriate value for your pack
7. Save Parameters to Flash
8. Try again

Precharge Failure Diagnosis Tree

```
System won't reach RUN mode
■
▼
Does opmode reach "precharge"?
■ ■
NO YES
■ ■
▼ ▼
Check: Does it then fail?
- Ignition ■
signal on ■■■ YES
Pin 15 ■
- Start signal ▼
on Pin 52 Check status field:
■
■■■■■■■■■■■■■■■■■■■■
■ ■ ■
"UDC below "din_break Other
```

```
udcsw" on"
■ ■ ■
▼ ▼ ▼
Check Brake Check shunt,
udcsw signal CAN wiring,
setting wiring udcmin
```

Throttle Status Errors

After precharge succeeds and `opmode` reaches `run`, the next common status message involves the throttle:

`Throttle 1 2` (or similar)

This means the VCU has a problem with one or both throttle channels. Common causes:

1. `potmode` **set to** `dual` **but only one channel wired** — Channel 2 reads a fixed value that disagrees with channel 1's movement
2. `pot2min`/`pot2max` **not calibrated** — Channel 2 plausibility check fails immediately
3. **Wiring fault** — One channel is open or shorted

1. Go to Spot Values
2. Watch `pot` and `pot2` while moving the pedal
3. Both should change. If one stays fixed, that channel has a wiring problem.
4. If both change but the system still faults, check that `pot2min`/`pot2max` are calibrated (not at defaults)

`din_break` — The Brake Signal Interlock

`din_break` is a critical safety interlock. **The VCU will not produce torque while** `din_break` **shows "on"**, regardless of throttle position or operating mode.

This causes significant confusion because the system appears to be in run mode, the throttle appears to work (potnom changes), but the motor does nothing.

- `din_break` = **off** → normal, torque is allowed
- `din_break` = **on** → brake is active, torque is blocked

Causes of `din_break` being stuck "on"

Cause	Fix
Brake wired to constant 12V	Check brake switch wiring — should be normally open
Brake switch wired normally-closed (NC)	Use the normally-open (NO) terminal
Pin 49 floating with electrical noise	Add 10kΩ pull-down resistor from Pin 49 to ground
Brake pedal held down during testing	Release the brake

Parameter Abbreviations Reference

The ZombieVerter interface uses abbreviated parameter names that can be confusing. Common ones:

Abbreviation	Full meaning
udc	Battery/pack voltage (U = voltage in German/Latin, DC = direct current)
udcsw	UDC switch — voltage at which precharge is considered complete
udcmin	UDC minimum — below this voltage, VCU will not enter run mode
udclim	UDC limit — maximum pack voltage before derating
invudc	Inverter-reported DC bus voltage
opmode	Operating mode
invstat	Inverter status
pot	Throttle channel 1 raw ADC value (pot = potentiometer, legacy name)
pot2	Throttle channel 2 raw ADC value
potnom	Normalised throttle demand (0–100)
din_break	Digital input — brake signal
din_forward	Digital input — forward direction signal
din_reverse	Digital input — reverse direction signal
din_start	Digital input — start signal
temphs	Temperature — heat sink (inside inverter)
tempm	Temperature — motor
idc	DC current from shunt
pch	Precharge (abbreviated in status messages)

Reading the Interface Efficiently

Use Ctrl+F

The web interface is a single long page. Use browser find (Ctrl+F) to locate specific parameters or spot values quickly. Search for `udcsw` to jump directly to that parameter.

Use Auto Refresh Strategically

Auto Refresh updates all spot values continuously. For diagnosis, turn it on and watch specific values while performing actions (turning ignition on, pressing start, moving throttle). For calibration, it's easier to turn it off and use the manual Refresh button so values don't jump while you're trying to read them.

Use the Visible Filter

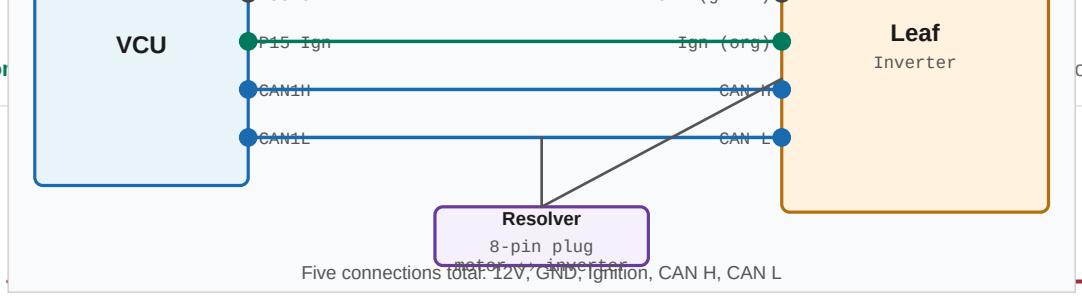
Check the Visible box for only the spot values you're currently watching, hide the rest. This dramatically reduces clutter, especially useful when diagnosing precharge issues (watch just `opmode`, `status`, `udc`) or throttle issues (watch just `pot`, `pot2`, `potnom`, `din_break`).

What's Next

- **C07** — Data Logging & Live Telemetry
- **W03** — Contactor & Precharge Circuit (for deeper precharge theory)

Source: "ZombieVerter VCU Won't RUN" — Good Enuff Garage (May 2024) | openinverter.org/wiki/ZombieVerter_VCU

Last verified against firmware: V2.30A (August 2025)



Five connections total: 12V, GND, Ignition, CAN H, CAN L

ure: Nissan Leaf Inverter — wiring overview

Overview

The Nissan Leaf inverter is the most common drivetrain in ZombieVerter-based conversions. It uses a CAN-based interface — straightforward wiring, well-documented, and supported across all three Leaf generations. The Leaf motor (EM57 or EM61) is a permanent magnet AC machine with excellent power density and broad speed range.

Leaf Generations

Generation	Pack	Motor	Inverter type	Notes
Gen1 (ZE0)	24kWh	EM57 (80kW)	Single inverter	2011–2017. Most common in conversions.
Gen2 (AZE0)	30/40kWh	EM57 (80–110kW)	Single inverter	2016–2022. Minor CAN differences.
Gen3 (ZE1)	40/62kWh	EM57/EM90	Revised inverter	2018+. Improved in V2.22A.

For conversion use, Gen1 and Gen2 are nearly identical from the VCU's perspective. Gen3 has some CAN differences that were addressed in V2.22A firmware.

Vehicle Connections

The Leaf inverter needs very few external connections:

Signal	Connection
Permanent 12V	12V+ to inverter (two gray wires — join together, add fuse)
`Ground`	Chassis ground to inverter (two gray wires — join together)
Ignition ON	12V when key is in ON position (orange wire from OEM harness)
CAN H	CAN bus high to VCU CAN1H
CAN L	CAN bus low to VCU CAN1L

That's five connections. The Leaf inverter is genuinely simple to wire.

> **Using the OEM harness:** If you have the Leaf wire harness, extract the inverter signal plug branch. If not, you can connect directly to the inverter connector pins — Damien's 2019 "Grey Goose" video documents exactly which pins carry which signals, including colour codes.

Resolver Wiring

The Leaf motor position is sensed by a resolver mounted in the motor. The resolver connects from the motor to the inverter via a separate harness plug (typically 8-pin).

The resolver carries: sine pair, cosine pair, exciter pair, and motor temperature sensor. All must be connected correctly or the inverter will report a resolver fault and refuse to run.

VCU Parameters

```
inverter = 1 (Leaf)
inverterCan = CAN1 (or CAN2 — try both if no communication)
```

The Leaf PDM (charger/DC-DC) can be added on the same CAN bus:

```
chargerType = LeafPDM
chargerCan = CAN1
```

Connectivity Check

With the Leaf inverter powered (12V applied, CAN connected, `inverter` = 1):

- Check `invstat` in Spot Values — should show a non-zero value when the inverter is communicating
- Check `temphs` — should show a plausible heatsink temperature (25–40°C at room temperature)
- If both read zero: check CAN assignment, check baud rate (should be 500 kbps), try swapping CAN1/CAN2 assignment

---

Minimum Pack Voltage

The Leaf inverter requires a minimum pack voltage to operate. Below approximately 200V the inverter will not run. This is hardcoded in the inverter firmware and cannot be changed from the VCU.

For bench testing at low voltage: use `udcsw` = 0 and set `udcmin` low to allow the VCU to attempt precharge. The inverter itself will refuse to produce torque below 200V — you will see the contactors sequence correctly but `invstat` will indicate the inverter is not ready.

The Leaf PDM — Charger and DC-DC

The Leaf Power Delivery Module (PDM) combines the AC charger and DC-DC converter. It uses the same CAN bus as the inverter and communicates using the same 500 kbps baud rate.

See Module A02 for full Leaf PDM charging setup.

Gen3 Specific Notes

The Gen3 ZE1 Leaf (2018+) has some differences in CAN message content that caused issues with early V2.x firmware. V2.22A (March 2025) addressed the Gen3 PDM specifically. If using Gen3 hardware, ensure firmware is V2.22A or later.

Known Issues and Gotchas

Common Build Context

The Leaf inverter/motor combination is physically an FWD transverse unit — compact and suited to front-wheel drive conversions. For RWD applications it requires a custom adapter to convert from the integrated FWD drive axle output to a propeller shaft or transfer case.

The motor + inverter typically weigh around 60kg combined. The motor alone is approximately 35kg.

Source: Damien Maguire @Evbmw — "The Grey Goose — Leaf Inverter Wiring" (July 2019) · "Nissan Leaf Motor Running" (2018)

openinverter.org wiki — Nissan Leaf inverter page

Last verified against firmware: V2.30A (August 2025)

TRACK INTEGRATION · MODULE I02

Tesla Drive Units (SDU & LDU)

Overview

Tesla drive units are increasingly popular in EV conversions — powerful, well-engineered, and becoming more available as early Model S vehicles reach end-of-life. The ZombieVerter controls Tesla drive units via an openinverter replacement logic board that installs in the inverter in place of the OEM Tesla logic board.

> **Critical distinction:** The openinverter board replaces the LOGIC board inside the inverter — not the entire inverter. The power electronics, motor, and housing remain OEM. Only the small control board that sits inside the inverter housing is replaced.

Small Drive Unit (SDU) vs Large Drive Unit (LDU)

	SDU	LDU
`Model`	Model S rear (pre-2016), Model 3 rear	Model S/X rear (2013+)
Peak power	~220kW	~375kW
`Weight`	~100kg	~145kg
`Layout`	Compact — suits smaller builds	Larger — needs more space
`Availability`	Good	Good and increasing
OI board version	V2.1	V2.1

Both use the same openinverter board and the same ZombieVerter integration approach.

The OpenInverter Board

The openinverter replacement board is available from evbmw.com or can be built from the GitHub files. It provides:

- CAN interface to the VCU
- Motor position encoding
- Gate drive signals to the IGBT modules in the power stage
- Replacement of all OEM Tesla logic functionality needed for EV conversion use

Set `inverter` = OIBoard (4) in VCU parameters.

Pre-Installation Checks — Do These Before Soldering Anything

Damien's 2023 video documents a systematic 8-step process. Inverter failures in Tesla conversions almost always result from skipping these steps. Follow them in order.

Step 1 — DC Bus Diode Test

With the inverter completely unpowered and disconnected, set multimeter to diode test mode. Connect positive probe to HV+ terminal, negative probe to HV– terminal. You should see: voltage rising from near zero as the capacitors charge, then climbing to open circuit. This confirms no dead short on the DC bus.

Reverse the probes (negative to HV+, positive to HV–). You should see approximately 0.6V — the forward voltage of the body diodes. This is correct.

> If you see a direct short (0V) in either direction: the inverter has failed IGBT modules. Do not proceed.

Step 2 — Phase Diode Tests

With multimeter negative on HV– bus, positive probe on each of the three motor phase terminals (U, V, W) in turn. Each should show increasing voltage then open circuit (capacitors charging). This confirms the upper IGBT body diodes are intact.

Reverse: negative probe on HV+, positive on each phase. Should see ~0.6V on each. This confirms the lower IGBT body diodes are intact.

Any phase showing a direct short = failed IGBT on that phase. Do not proceed.

Step 3 — 12V Logic Board Test (OEM board still installed)

Apply 12V power to the logic board connector only — do not connect HV. The board should draw approximately 200–300mA and show some activity (LEDs, CAN frames on a bus monitor). If the board draws no current or draws excessive current, investigate before proceeding.

Step 4 — Solder the OpenInverter Board

Remove the OEM logic board. Solder the openinverter board connectors carefully — cold joints on the gate driver connections cause intermittent or failed gate switching. Inspect under magnification before proceeding.

Step 5 — 12V Test With OI Board

Apply 12V to the logic board connector with the OI board installed. The board should power up, CAN should be active, no excessive current draw.

Step 6 — Install Board Into Inverter

Reinstall the cover. The board is now installed but HV has not been applied yet.

Step 7 — 12V Test With OI Board Installed in Inverter

Apply 12V again — verify normal operation with the board in its installed position.

Step 8 — Encoder Test Before Any HV

With the VCU connected, CAN active, and 12V applied: rotate the motor shaft slowly by hand. Watch encoder or resolver readings in the VCU Spot Values. The values should change smoothly as the shaft rotates. If they don't — investigate the encoder wiring before applying HV.

> **Only after completing all 8 steps should you apply HV to the drive unit.**

Why These Steps Matter

Damien's 2023 video was specifically created in response to a pattern of inverter failures in the community. The failures were caused by:

1. **Applying HV before the DC bus was verified** — a pre-existing fault that would have been caught by the diode test
2. **Applying HV before the encoder was verified** — the inverter attempts motor commutation immediately on HV application; if encoder data is wrong it commands incorrect phase currents and destroys the power stage

The OI board itself is not expensive. The IGBT modules in the inverter power stage are expensive and difficult to source. The 8-step process is 30 minutes of work that prevents a catastrophic and expensive failure.

HVIL on Tesla Drive Units

Tesla drive units have an HVIL connector that must be looped for the inverter to operate. The HVIL connector carries a low-current loop through the HV connector housing — if the HV cover is removed, the loop breaks.

In a conversion, the HVIL connector must either be looped (a simple wire jumper connecting both pins) or integrated into the vehicle's HVIL circuit. Leaving it unconnected will prevent the inverter from entering run mode.

Cooling Requirements

Tesla drive units are liquid cooled. In the OEM vehicle, they use Tesla's proprietary coolant loop with a pump, radiator, and chiller. In a conversion, you need an aftermarket electric coolant pump, a radiator or heat exchanger, and thermostat control.

The VCU can control a coolant pump via the `CoolantPump` output function and a fan via `CoolingFan` with the `FanTemp` threshold parameter (added V2.20A).

Minimum coolant flow should be maintained whenever the inverter is active. Unlike the GS450h which uses oil, the Tesla drive unit uses water/glycol coolant.

VCU Parameters

```
inverter = 4 (OpenInverter board)
inverterCan = CAN1
```

Community Resources

The Volvo V50 Tesla Model 3 drive unit project (2025) is the most recent documented Tesla LDU conversion with ZombieVerter. Three build videos are available on Damien's channel documenting the integration process with current firmware.

Source: Damien Maguire @Evbmw — "Using Tesla Drive Units with OpenInverter Board" (July 2023) · "Volvo V50 Tesla Model 3 Drive Unit" series (2025)

Last verified against firmware: V2.30A (August 2025)

TRACK INTEGRATION · MODULE I03

GS450h Transaxle

Why the GS450h is Different

The Lexus GS450h (and GS300h) transaxle is one of the most popular choices for rear-wheel-drive EV conversions — particularly for classic American muscle cars, trucks, and anything with a traditional longitudinal drivetrain layout. It is, however, the most mechanically complex integration the ZombieVerter supports.

Key differences from Leaf/Outlander/Prius integrations:

Feature	Leaf / Outlander	GS450h
Drive unit	Transverse FWD	Longitudinal RWD
Motor count	1 traction motor	2 motors (MG1 + MG2)
Reverse gear	Software (negative torque)	Physical gear in transmission
Physical shifter	Not needed	Required (Park/Reverse/Neutral/Drive)
Inverter comms	CAN bus	Synchronous serial (8-wire)
`Gearbox`	None (single fixed ratio)	Two-speed (high ≈ 2:1, low ≈ 4:1)

The Two Motors: MG1 and MG2

The GS450h contains two motor-generators:

Motor	Role
MG1	Generator / starter — primarily used for engine starting and power split in the OEM hybrid system
MG2	Traction motor — the primary drive motor in an EV conversion

In a ZombieVerter conversion, MG2 is used for traction. MG1 may or may not be utilised depending on the conversion.

Synchronous Serial Interface

Unlike the Leaf (CAN) and Outlander (CAN), the GS450h inverter uses a **synchronous serial bus** — an 8-wire interface specific to Toyota/Lexus systems. This is also used for the Toyota Prius Gen3 and the Lexus GS300h.

- The VCU's CAN buses are not used for inverter communication with GS450h
- The sync serial wires connect to specific pins on the ZombieVerter's 56-pin connector
- The wiring is more complex than CAN (8 wires vs 2)

Sync Serial Pin Assignments (ZombieVerter)

The four synchronous serial pairs connect to the following VCU pins:

Signal pair	VCU Pin (–/negative)	VCU Pin (+/positive)	Notes
REQ	16	17	Request
CLK	18	19	Clock
MTH	20	21	Motor to hybrid (inverter → VCU)
HTM	22	23	Hybrid to motor (VCU → inverter)

Pin assignments may vary slightly between firmware versions and board revisions — always verify against the current wiki pinout diagram.

Inverter Connector

The GS450h inverter connector is a large multi-pin connector (approximately 36 pins) that combines:

- The synchronous serial interface (8 pins used)
- Power (12V supply — 2 wires redundant positive, 2 wires redundant ground)
- Resolver connections - the resolver signals for MG1 and MG2, used for motor position sensing. This is **OEM harness territory** — use the original Toyota/Lexus wire harness wherever possible. Building resolver cables from scratch is possible but requires shielded twisted pairs and careful attention to wiring.

Power Wiring to the Inverter Connector

Wire colour (OEM)	Function	Notes
Green (×2)	12V positive supply to inverter	Two wires, combined — add an inline fuse
White/black (×2)	Ground	Two wires, combined

The redundant positive wires run from the connector back to a common point and rejoin — they are Toyota's built-in redundancy. In an EV conversion, join them together and run to a fused 12V supply.

Start with a 1A fuse and work upwards if it blows (2A, 3A, 5A). The inverter draws more than the VCU at startup.

Wire Harness — OEM vs. DIY

Use OEM Harnesses Where Possible

The synchronous serial wiring is particularly important to get right. Toyota's harness uses:

- Twisted shielded pairs for the sync serial signals
- Specific loop-back connections within the connector (wires that exit and re-enter the same connector)
- Shielding for EMI protection

Building these from scratch is achievable but error-prone. Buying the OEM harness and extracting only what you need is strongly recommended.

What to Extract from the GS450h Harness

The GS450h "engine wire" harness (also called the transmission wire harness) contains everything you need but also a lot you don't need. A typical harness extraction yields:

What you extract	Used for
MG1 resolver cable (4-pin or 10-pin connector)	Motor position sensing
MG2 resolver cable	Motor position sensing
Shift solenoid harness (SL1, SL2)	Transmission gear control (high/low)
Neutral safety switch harness	Park/Reverse/Neutral/Drive detection

You'll only use approximately 12% of the total harness — the rest is OEM engine/hybrid control wiring that can be discarded.

Harness Identification Tips

When buying a used GS450h harness:

- Look for the "squiggly" plastic guide that runs along the transmission side
- The resolver connectors are at one end — MG1 and MG2 each have their own plug
- The resolver plugs are at the **opposite end** from the main inverter connection — if a harness is cut at the wrong place, you lose the resolver plugs
- The neutral safety switch plug is gray with plastic shielding, on a branch that includes many ground wires
- Gen 3 (2007–2011) and Gen 4 (2013+) harnesses differ — Gen 3 is the supported conversion platform as of current firmware

Resolver Cable Identification

On a 4th gen harness:

- MG2 resolver: gray/green wires + temp sensor wires (red/blue)
- MG1 resolver: separate plug, typically 4 or 10 pins
- Temp sensor wires must be cut (they don't have through-plugs, only connect on one end)

Shift Solenoids

The GS450h has two speeds — high gear (~2:1 ratio) and low gear (~4:1 ratio). The shift solenoids SL1 and SL2 control which gear the transmission is in. In ZombieVerter, these are controlled via:

VCU pin assignment	Function
Trans SL1–	Transmission solenoid 1 ground-side control
Trans SL2–	Transmission solenoid 2 ground-side control

For basic operation, many builders leave the transmission in high gear and don't connect the shift solenoids. The zombie does have parameters for managing gear shifts, but this requires more advanced configuration.

Gear Shifter Switch

The gear shifter switch tells the VCU which gear position the physical shifter is in (Park, Reverse, Neutral, Drive). The switch physically activates the parking pawil when in PARK, all other positions are simply an electrical switch. This is wired to the ZombieVerter which reads the switch and responds accordingly — it will not send torque in Park or Neutral.

For bench testing without the full shifter assembly, specific pins on the Gear Shifter switch can be shorted to simulate a Drive signal and allow torque commands. Refer to the openinverter wiki for the specific pins.

VCU Parameter Settings

Parameter	Value for GS450h
`inverter`	`2` (GS450H)
`vehicle`	Match your donor vehicle (Classic, BMW E46, etc.)
`transmission`	`auto` or per your shifter setup
`gearLever`	Match your gear selector type

Oil Pump

The GS450h inverter has its own oil cooling circuit with an electric oil pump. The ZombieVerter controls this pump via:

VCU function	Type	Notes
`GS450Hpump`	PWM OUTPUT	Controls oil pump speed proportionally

Assign `GS450Hpump` to the `Pump PWM` pin in the configurable output assignments. The pump must run when the inverter is operating to provide oil pressure to the transmission to allow traction

Bench Testing the Sync Serial Connection

A basic connectivity check without HV:

1. Connect sync serial wires (8 wires, pins 16–23) from VCU to inverter connector
2. Connect 12V power to inverter connector (green wires + ground)
3. Apply permanent 12V to VCU (Pins 55, 56)
4. Connect to web interface
5. Go to Spot Values — check `temphs` (heat sink temperature)

If `temphs` shows a non-zero, plausible temperature value (e.g., 25–35°C at room temperature), the sync serial connection is working and the inverter is communicating. If it shows 0 or negative, the sync serial is not connected correctly.

> Note: The inverter will draw over 1A from the 12V supply at startup. If using a bench power supply, ensure it is set for at least 2A output.

Common GS450h Integration Issues

Issue	Cause	Fix
No temp reading from inverter	Sync serial not connected or wrong pins	Re-verify pin assignments against current wiki pinout
Ground loop / noise interference	Laptop USB connected to logic analyser during same test	Disconnect USB/logic analyser, check chassis grounds
System won't stay in run after restart	ISA shunt on ignition-switched power	Move shunt to permanent power supply
Resolver errors	Swapped MG1/MG2 connectors	Swap resolver plugs

What's Next

- **I04** — Mitsubishi Outlander PHEV Drive Unit
- **A01** — Regen Tuning (GS450h has specific regen considerations)

Source: "ZombieVerter VCU Prius Inverter Connection" series — Good Enuff Garage (May 2024) | "2013 Lexus GS450h Transmission Wire Harness" — Good Enuff Garage (Jun 2025) | openinverter.org wiki

Last verified against firmware: V2.30A (August 2025)

TRACK INTEGRATION · MODULE I03-X

GS450h — Wiring & Oil Pump Deep Dive

Overview

This module covers the three most critical details of a GS450h integration that are most often done wrong: the HV connection bypass, the sync serial wiring, and the oil pump controller setup.

HV Connection — The Critical Bypass

The GS450h inverter has two places you can connect HV. Only one is correct.

The OEM Terminals — DO NOT USE

Connecting HV to the OEM positive/negative terminals routes current through the boost inductor and boost IGBT. These components are rated for approximately 100A maximum. Everything works fine on the bench and at low speed. Under sustained load at highway speed, the inverter destroys itself. At least two confirmed inverter failures from this mistake are documented in the community.

The Brass Pillars — Correct Connection

Two brass pillars are accessed by removing a small black plastic cover and two M6 bolts (10mm head). These connect HV directly to the motor drive stage, bypassing the boost converter entirely.

1. Locate the black plastic cover on the inverter body
2. Gently prize off the cover — Toyota uses adhesive
3. Remove the two M6 bolts (10mm head)
4. HV+ to one pillar, HV– to the other

For bench testing: A 60V 5A power supply into the brass pillars is sufficient to verify sync serial communication and low-speed rotation. You do not need full pack voltage for initial bench testing.

Sync Serial Interface — 8 Wires, 4 Differential Pairs

Unlike the Leaf (CAN) and Outlander (CAN), the GS450h uses a synchronous serial bus.

Pin Assignments (ZombieVerter VCU)

Signal	Direction	VCU Pin (–)	VCU Pin (+)
REQ	VCU → Inverter	16	17
CLK	VCU → Inverter	18	19
MTH	Inverter → VCU	20	21
HTM	VCU → Inverter	22	23

Always verify against the current wiki pinout. Pin assignments may vary between board revisions and firmware versions.

Cable Requirements

- Use shielded twisted pair for each differential pair
- Toyota's OEM harness uses shielded twisted pairs — replicate this for reliable operation
- Building from scratch is achievable but error-prone; using the OEM harness is strongly recommended

Connectivity Check (No HV Required)

With sync serial correctly wired and 12V applied to both VCU and inverter:

- Check `temphs` in Spot Values
- A reading of approximately 25–35°C at room temperature confirms the sync serial link is working
- A reading of 0 or negative means the sync serial is not communicating

---

12V Power to Inverter

From the OEM harness:

OEM wire colour	Function
Green (×2)	12V+ supply to inverter — join both, add fuse
White/black (×2)	Ground — join both

Start with a 1A fuse and work upwards if it blows (2A → 3A → 5A). The GS450h inverter draws more than the VCU at startup.

The 12V supply to the inverter is switched by the VCU via the inverter enable relay (Pin 32). The inverter should not power up until the VCU is ready.

Resolver Cable Extraction

The OEM "engine wire" harness contains the sync serial cables, resolver cables, and other signals. You use approximately 12% of the total harness.

What to Extract

Cable	Used for
MG1 resolver cable	Motor position sensing
MG2 resolver cable	Motor position sensing
Neutral safety switch harness (9-wire gray plug)	P/R/N/D detection
Shift solenoid harness (SL1, SL2)	Gear shift control (if used)

Key Tips

- **Blade technique:** Cut electrical tape and loom lengthwise with a sharp blade. A dull blade cuts wire insulation. Replace the blade before starting.
- **MG2 plug is at the rear of the transmission, MG1 is at the front.** Label them immediately on extraction — swapped resolver plugs cause resolver errors and the system won't run correctly.
- **Temperature sensor wires must be cut.** MG1 and MG2 temperature sensor wires (gray/green on MG2, red/blue on MG1) run inside the resolver cable bundle but terminate at the "harmonica" junction block and cannot be used. Cut them as far back as possible.

· **Gen 3 vs Gen 4:** Gen 3 (2007–2011) is the primary supported conversion platform. Gen 4 (2013+) harnesses may be physically compatible but test first.

Buying a Harness

Search eBay for "GS450h engine wire" or "GS450h transmission harness." Expect to pay $125–200. Make lowball offers — sellers often accept. Always confirm from photos that the resolver plugs at the far end are not cut — this is the most commonly damaged part of a used harness.

Oil Pump System

The GS450h inverter has its own internal oil cooling circuit. An electric oil pump must run continuously while the inverter is active — there is no mechanical pump drive in an EV conversion.

Why It Matters

If the oil pump is not running when the inverter is active, the inverter overheats. This is not a gradual problem — the pump must run from precharge, throughout run mode, and should not stop until the inverter is fully de-energised.

Oil Pump Specifications

Parameter	Value
Current draw	~25A from 12V at full speed
Power source	Dedicated 12V — NOT from VCU output pin
Control method	External controller with PWM input from VCU
Starting point	30% PWM duty cycle

Oil Pump Controller Wiring

Wire colour	Function
`Blue`	12V+ supply to controller — 30A fuse minimum
`Brown`	Ground to controller casing
`Black`	PWM signal from VCU (GS450Hpump function)

The controller has two connectors: power (to the pump motor) and resolver (pump position feedback).

■ **The ~25A draw cannot come from a VCU output pin.** These are low-current signal outputs. Wire the controller power directly from the 12V battery or auxiliary bus with its own dedicated fuse.

VCU Configuration

In Parameters → Dout, assign the `GS450Hpump` function to the Pump PWM output pin. Start at 30% duty cycle — this is sufficient for low-load bench operation. Increase for sustained high-load or elevated ambient temperatures.

From V2.20A, the PumpPWM output function is freely assignable — not limited to GS450h builds. This allows coolant pump PWM control on other drivetrains using the same pin assignment method.

Input Shaft Lock

Without locking the input shaft, MG1 can only spin the engine input shaft — it cannot send torque to the output. The lock forces MG1 torque through the power split device to the output shaft.

Without the lock, you lose approximately half your available torque and power. MG1 and MG2 combined with a properly locked shaft give a torque multiplication through the planetary gearset of approximately 2.6–2.7:1 for MG1's contribution.

The lock is a mechanical plate or bar with a spline fitting that bolts to the bellhousing flange and engages the engine input shaft splines. For bench testing only, Damien uses a vice grip and cable tie. For a real vehicle, a purpose-made billet lock plate is required.

Common Integration Issues

Issue	Cause	Fix
No temphs reading (0 or negative)	Sync serial not connected or wrong pins	Re-verify pin assignments against current wiki pinout
Ground loop / noise interference	Laptop USB connected to logic analyser during test	Disconnect USB/logic analyser, check chassis grounds
System won't stay in run after restart	ISA shunt on ignition-switched power	Move shunt to permanent power supply
Resolver errors	Swapped MG1/MG2 resolver connectors	Swap the resolver plugs
Inverter destroyed under load	HV connected to OEM terminals instead of brass pillars	Replace inverter, connect to brass pillars only

Source: Damien Maguire @Evbmw · Good Enuff Garage · openinverter.org/wiki

Video sources: "GS450h EV Drivetrain Deep Dive" (2020) · "GS450h and GS300h VCU Connection" (2022) · "GS450h Transmission Wire Harness" (2025)

Last verified against firmware: V2.30A (August 2025)

TRACK INTEGRATION · MODULE I04

Mitsubishi Outlander Drive Unit

Overview

The Mitsubishi Outlander PHEV contains separate front and rear drive units — each with its own motor, inverter, and three-phase connection. Either or both can be used in a conversion. The interface is CAN-based, similar to the Nissan Leaf but with Mitsubishi-specific message IDs.

> **Note on versions:** Early firmware used `inverter` = 6 for Outlander, which has since been deprecated. Use `inverter` = 8 for the rear unit. Check the current wiki for the front unit value as this has changed across firmware versions.

Front vs Rear Drive Unit

The Outlander PHEV has distinct front and rear units:

	Front	Rear
Motor type	PMAC	PMAC
Peak power	~60kW	~60kW
`Orientation`	Transverse FWD	Longitudinal RWD
`Interface`	CAN	CAN
ZV support	Partial — check wiki	Full — V2.15A+
Common use	Front-wheel drive conversions	Rear-wheel drive conversions, longitudinal builds

The rear unit is the more commonly used in ZombieVerter conversions and has more complete firmware support.

Minimum Voltage

> **The Outlander inverter requires a minimum of approximately 200V to operate.** Below this threshold the inverter will not produce torque. This is hardcoded in the inverter and cannot be changed from the VCU.

For bench testing: use a bench power supply at a low voltage to verify CAN communication. The inverter will communicate below 200V — it just won't spin. You need pack voltage to verify full operation.

Wiring

The Outlander inverter needs:

Signal	Notes
12V+ permanent	To inverter logic power
`Ground`	Chassis ground
CAN H / CAN L	500 kbps, VCU CAN1 or CAN2

Signal	Notes
HV+ / HV–	Via contactors and precharge as normal
Motor three-phase	Three-phase AC to motor (integral in most units)
`Resolver`	Motor position sensor — OEM harness or equivalent

The motor and inverter are typically inseparable in the Outlander unit — the motor is integrated into the transaxle housing and the inverter bolts directly on. You take them together from the donor vehicle.

The Outlander OBC — Two 12V Feeds

The Outlander OBC (onboard charger) is a separate unit that provides both AC charging and DC-DC conversion. It is covered in Module A02.

The most common Outlander build failure: wiring only one of the two 12V power inputs to the OBC. Both must be connected. See A02 for full OBC wiring.

OEM Harness

For the Outlander drive unit, obtain as much of the OEM harness as possible. The resolver connector and cooling system connectors have non-standard pinouts that are significantly easier to work with if you have the original plugs.

Mitsubishi forum resources (as distinct from openinverter) can be valuable for Outlander-specific pinout information — Damien noted he sourced the initial CAN log data that enabled Outlander support from a Mitsubishi enthusiast forum.

Cooling

The Outlander drive unit requires active liquid cooling. The OEM system uses a dedicated inverter coolant circuit separate from the engine coolant. In a conversion:

- Electric coolant pump required (12V, controllable)
- Assign `CoolantPump` to a VCU digital output pin
- The pump should run whenever the inverter is active — assign to activate in precharge and run mode

The Outlander water heater (cabin heating) is also supported by the ZombieVerter and communicates via CAN. Separate VCU parameter assignment required.

CAN Communication Verification

With 12V applied and CAN connected:

- Check `invstat` in Spot Values — should show non-zero when inverter is communicating
- Check `temphs` — should show a plausible temperature
- If no communication: verify `inverter` parameter is set correctly, check CAN bus assignment, verify 500 kbps baud rate, check twisted pair CAN wiring with 120Ω termination at both ends

Known Issues

Outlander front unit CAN parameter value changed between firmware versions. Always verify against the current openinverter wiki before setting this for the front unit — the deprecated value 6 (used before V2.17A) will not work correctly on current firmware.

Rear unit bug fixed in V2.17A (May 2024). If using the rear unit, firmware must be V2.17A or later. Earlier firmware had a bug that prevented correct rear unit operation.

Source: Damien Maguire @Evbmw — "Mitsubishi Outlander Front Drive Unit" (May 2021) · openinverter.org forum

Rear Outlander support: V2.15A · Rear unit bug fix: V2.17A · Old inverter=6 deprecated

Last verified against firmware: V2.30A (August 2025)

TRACK ADVANCED · MODULE A01

Regen Tuning

Important Firmware Note

> **Regen was not reliably implemented until V2.15A (April 2024).** Any content — videos, forum posts, tutorials — about ZombieVerter regen from before April 2024 refers to a non-functional or experimental implementation. Do not follow pre-V2.15A regen instructions.

>

> V2.20A (December 2024) added `revRegen` — control of regen behaviour in the reverse direction.

How Regen Works on the ZombieVerter

Regen reverses the normal energy flow. Instead of the battery powering the motor, the motor acts as a generator — converting kinetic energy (vehicle momentum) back into electrical energy and pushing it into the battery.

The VCU triggers regen by sending a **negative torque command** to the inverter. From the inverter's perspective, it's the same as a forward torque command but in the opposite direction. The inverter's control loops handle the actual energy reversal.

There are two distinct regen mechanisms:

Throttle-lift regen — triggered by lifting the throttle. When `throtmin` is set to a negative value (e.g. -5), lifting the throttle produces a light braking effect rather than pure coast.

Brake regen — triggered by the brake pedal signal. Additional regen on top of (or instead of) throttle-lift regen when the brake is pressed.

The Key Parameters

throtmin — Throttle-Lift Regen

throtmin value	Effect
0 (default)	No regen on throttle lift — pure coast
-5	Light one-pedal style regen
-20	Moderate regen — noticeable engine-brake feel
-50	Strong regen — aggressive deceleration on throttle lift

Start conservative (e.g. -5) and increase gradually. Aggressive regen values feel unnatural and can be unsettling for passengers who aren't expecting strong deceleration on every throttle lift.

> Damien's personal preference for his E39 GS450h build: "I'm not a huge fan of regen myself" — but notes that many builders want it for practical range extension and one-pedal driving feel.

brakelight output

When regen is active — either from throttle lift or brake pedal — the VCU can activate the brake lights via an assigned output pin. Assign the `BrakeLight` output function to a digital output pin in the Dout section. Without this, regen deceleration happens without brake lights — a safety concern.

revRegen (V2.20A+)

Controls whether regen is active in reverse direction. Some builds need regen in reverse (regenerative downhill reverse), others don't. Toggle on or off.

Drivetrain-Specific Regen Behaviour

Regen behaviour varies by inverter type. This is why pre-V2.15A regen was unreliable — each drivetrain handles negative torque commands differently, and the VCU firmware needed per-drivetrain tuning.

Tuning Process

Step 1 — Start with throttle-lift only

Set `throtmin` = -5. Drive at moderate speed and lift the throttle gently. You should feel a light braking effect. Watch `idc` in Spot Values (via OBD or web interface) — it should go negative (current flowing back into battery) when regen is active.

If `idc` stays at zero or positive: regen is not working. Check firmware version is V2.15A+, check that `throtmin` is saved to flash, verify that `throtmax` is not zero.

Step 2 — Increase gradually

Increase `throtmin` by -5 increments. At each step, test the feel:

- Does the deceleration feel proportional to throttle lift?
- Does it come on smoothly or with a jerk?
- Does it feel predictable to a passenger in the back seat?

Stop when it feels natural. Most builders settle between -10 and -30.

Step 3 — Verify brake lights

Plug in a brake light tester or have someone observe while you trigger regen. Brake lights must illuminate whenever regen is actively decelerating the vehicle.

Step 4 — Check BMS limits

The BMS charge current limit caps how much regen current can flow back into the battery. If `idc` during regen hits the BMS charge limit and flattens, the inverter is limiting regen to protect the battery — this is correct behaviour. You cannot tune past this limit without raising the BMS charge current setting.

Common Regen Problems

Symptom	Likely cause	Fix
No regen at all	Firmware < V2.15A	Update firmware

Symptom	Likely cause	Fix
No regen at all	throtmin = 0	Set to negative value and save to flash
Jerky/harsh regen onset	throtmin too negative	Reduce magnitude (closer to 0)
Regen cuts out unexpectedly	BMS charge limit hit	Raise BMS max charge current if battery allows
Brake lights not on during regen	BrakeLight output not assigned	Assign BrakeLight function to a Dout pin
Regen feels inconsistent	Dual-pot mismatch triggering plausibility fault	Recalibrate both throttle channels
GS450h drivetrain jerk on regen	Auto gear shift timing	Expected behaviour — V2.15A downshifts before regen

One-Pedal Driving

With `throtmin` set to an appropriate negative value, the vehicle can be driven almost entirely with the throttle pedal — lift for regen deceleration, press for acceleration. Friction brakes are used only for final stops.

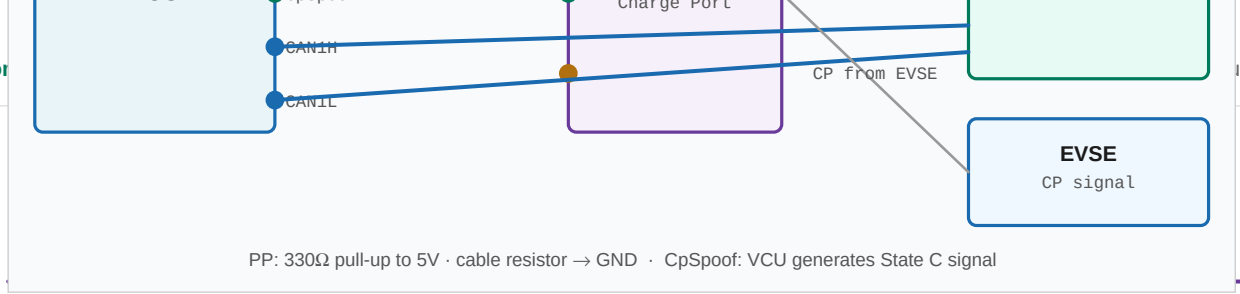
This style of driving:

- Increases range by recovering kinetic energy
- Reduces brake wear significantly
- Requires a learning period — new drivers should be warned about the strong deceleration on throttle lift
- Is most effective on routes with repeated stop-start patterns (urban driving)

Source: Damien Maguire @Evbmw — "ZombieVerter VCU Firmware 2.15" (April 2024) · "ZombieVerter VCU Software V2.04" (January 2024)

Regen first working: V2.15A (April 2024) · revRegen added: V2.20A (December 2024)

Last verified against firmware: V2.30A (August 2025)



PP: 330Ω pull-up to 5V · cable resistor → GND · CpSpoof: VCU generates State C signal

rge Control & Popular Chargers — wiring overview

Overview

The ZombieVerter supports AC onboard charging, CHAdeMO DC fast charging, and CCS via the BMW i3 LIM or Foccci open-source controller. One VCU can manage any of these, and combinations are possible (e.g. Leaf PDM for AC plus CHAdeMO for fast charging in the same vehicle).

Supported Chargers

Charger	AC Power	Notes
Nissan Leaf PDM Gen1	3.3 kW	Simplest setup — no CP/PP wiring to VCU
Nissan Leaf PDM Gen2	6.6 kW	Recommended first build
Nissan Leaf PDM Gen3	6.6 kW	2018+ Leaf. Improved in V2.22A
Mitsubishi Outlander OBC	3.3 kW	AC + DC-DC. Two 12V feeds required.
VW Golf PHEV OBC	3.3 kW	VAG protocol. Two CAN buses.
Audi e-tron OBC	7.2–11 kW	Most popular VW option.
MG ZS Gen1 OBC	6.6 kW	VAG protocol.
MG ZS Gen2 OBC	6.6–11 kW	3-phase capable.
Tesla Gen2 charger	10 kW	Requires open-source controller board.
Tesla Gen3 charger	18.5 kW	Requires Gen3 v2 controller board.
`Elcon`	1.5–6.6 kW	Universal J1772 charger.
`CHAdeMO`	DC 50+ kW	Native VCU support. 6 PCB jumpers required.
CCS (BMW i3 LIM)	DC fast	Salvaged from i3. Type 1 and 2.
CCS (Foccci)	DC fast	Open-source controller. V2.20A one-click setup.

Understanding CP and PP Signals

Every J1772 / Type 2 AC charging setup uses two signals from the charge port to the VCU. Understanding these is essential before wiring any charger except the Leaf PDM.

Control Pilot (CP)

The EVSE (charge station) generates the CP signal. It tells the car how much current is available and detects when the car is ready to charge. The car signals readiness by pulling down the pilot voltage through a resistor.

State	CP voltage	Meaning
A	+12V DC	No vehicle connected
B	+9V DC	Vehicle present, not ready
C	+6V PWM	Vehicle ready — charging permitted
D	+3V PWM	Ready + ventilation required

The PWM duty cycle encodes available current: 10% = 6A · 25% = 15A · 50% = 30A.

Proximity Pilot (PP)

PP detects whether a cable is physically plugged in. A resistor inside the cable connector pulls down a 5V sense line. Different cable current ratings use different resistor values.

PP resistor in cable	Cable rating
Open circuit	No cable / disconnected
1.5 kΩ	13A cable
680 Ω	20A cable
220 Ω	32A cable

Leaf PDM exception: The Leaf PDM detects cable insertion internally via its own CP circuit. You do not need to wire PP to the VCU for a Leaf PDM build.

Charge Mode Operation

Charge mode is triggered automatically when the VCU detects cable insertion via PP/ProxPilot, or a CHAdeMO/CCS handshake begins.

What Changes in Charge Mode

Aspect	In charge mode
opmode	Shows "charge"
HV contactors	Closed — HV bus live for charging
Throttle commands	Ignored — zero torque to inverter
DC-DC converter	Active — charges 12V battery from HV
Charge current	VCU sends power/current command to OBC via CAN

Key Charge Parameters

Parameter	What it does
chargerType	Selects your OBC (LeafPDM, Outlander, VAG, Tesla…)
chargePower	Watts requested from OBC (Leaf PDM)
chargeCurrent	Max AC current limit (A)
chargeVoltage	Target / maximum pack voltage

Parameter	What it does
`PPthreshold`	PP ADC value that triggers charge mode
`chargeStart` / `chargeStop`	Scheduled charge timer (hour of day)

Setup Checklist

1. Set `chargerType` — save to flash
2. Wire PP if required (not Leaf PDM): 330Ω pull-up to 5V → analogue input → assign ProxPilot → calibrate PPthreshold
3. Wire CpSpoof if required: assign to PWM output. Needed for Outlander, Foccci, most VW OBCs
4. Set `chargeVoltage` and `chargeCurrent` limits
5. Test: plug in cable → watch `opmode` in Spot Values → should transition to "charge" → watch `idc` for current flow

---

Nissan Leaf PDM

The simplest charging setup. The PDM combines the AC charger and DC-DC converter in one unit.

```
chargerType = LeafPDM
chargePower = 1000 ← start at 1kW, ramp up
chargePower = 3300 ← Gen1 max
chargePower = 6600 ← Gen2/3 max
chargePower = 0 ← stop charger
```

DC-DC note: The PDM's DC-DC activates in charge mode. Ensure permanent 12V on VCU — a 12V dropout reboots the VCU mid-session.

Mitsubishi Outlander OBC

3.3 kW AC charger + DC-DC converter. Supported since V2.01A.

■ **Two 12V power inputs — both must be fed.** The Outlander OBC has two separate 12V connectors. Both must be powered. Wiring only one: `opmode` shows "charge" but `idc` = 0 and no current flows. This is the most common Outlander charging failure.

CP/PP wiring required. From V2.20A, use `CpSpoof` PWM output for CP. Wire PP with 330Ω pull-up to 5V.

V2.20A also added a required CAN heartbeat message — without it the OBC stops after a timeout. Current firmware handles this automatically.

VW / Audi OBC and MG ZS

All use VAG protocol — set `chargerType` = VAG.

■ **Two separate CAN buses required.** VW chargers use "Hybrid CAN" and "Powertrain CAN" internally. Both must connect to different VCU CAN ports. Wire only one bus and the charger ignores you.

Audi e-tron OBC (7.2–11 kW): Most popular VW option. The VCU sends startup CAN commands that disable OEM requirements for charge port lock feedback and LIN features — no OEM Audi charge port needed. Community credit: Mitch Elliott's reverse engineering.

MG ZS Gen2 (11 kW): 3-phase capable. Also VAG protocol.

All VW/MG chargers need PP wiring (330Ω pull-up) and CpSpoof or physical CP circuit.

Tesla Gen2 and Gen3 Chargers

Both require dedicated open-source controller boards — available from evbmw.com or build from GitHub.

Both need a standard Type 2 charge port. PP sense → VCU ProxPilot analogue input (330Ω pull-up). CP is generated by the controller board.

Tesla Gen2 DCDC converter also supported as of V2.04A.

CHAdeMO DC Fast Charging

Native VCU support — no extra controller board needed, but the PCB must be configured first.

PCB Configuration — 6 Solder Jumpers

On the PCB rear silk screen, solder:

- **SJ6, SJ7, SJ8, SJ9** — enables HS CAN3
- **2 jumpers above R31** — adds fault-tolerant termination

Without all 6, CHAdeMO handshake will not complete.

12V Signal Wiring

CHAdeMO socket pin	Connect to
Pin 1	12V chassis GND
Pin 2	Contactor coil + (both coils paralleled)
Pin 4	VCU GP Out 3 (charge enable/disable)
Pin 10	Contactor coil – (both coils)
CAN H/L	CAN3 on VCU

CHAdeMO does not need PP — handshake is fully CAN + 12V signal based.

CCS — BMW i3 LIM or Foccci

Method	Notes
BMW i3 LIM	Salvaged from i3. Provides CCS Type 1 and 2. Connect via ZV Comms CAN parameters.
`Foccci`	Open-source CCS controller. V2.20A one-click setup. Default CAN Node ID = 22. Uses CpSpoof PWM from VCU.

Charger Selection Guide

Charger	Power	Complexity	CP/PP to VCU	Best for
Leaf PDM Gen2	6.6 kW	Low	Not needed	First build, Leaf stack
Outlander OBC	6.6 kW	Medium	PP + CpSpoof	Outlander builds
VW Golf PHEV OBC	3.3 kW	Medium	PP + CP	European builds, cheap
Audi e-tron OBC	7.2–11 kW	Medium	PP + CpSpoof	High-power AC
MG ZS Gen2 OBC	11 kW	Medium	PP + CpSpoof	3-phase 11kW
Tesla Gen2	10 kW	High — controller board	Full charge port	High AC power
Tesla Gen3	18.5 kW	High — controller board	Full charge port	Maximum AC speed
CHAdeMO	DC 50+ kW	High — PCB jumpers + wiring	Not needed	Public fast charging

Recommended first build: Leaf PDM Gen2. Most documented, no PP wiring to VCU, fewest gotchas.

Source: Damien Maguire @Evbmw · Good Enuff Garage · openinverter.org

Last verified against firmware: V2.30A (August 2025)

TRACK ADVANCED · MODULE A03

BMS Integration

Overview

The ZombieVerter does **not** implement cell-level BMS functions. It reads pack-level data from a separate BMS via CAN and enforces current limits from that BMS. This is an intentional division of labour:

- **ISA shunt** — pack-level monitoring (voltage, current, SoC). Feeds precharge logic and contactor control.
- **Cell BMS (SimpBMS, Orion, etc.)** — cell-level monitoring, balancing, per-cell protection. Sends charge/discharge limits to VCU over CAN.
- **VCU** — enforces BMS limits, manages contactors, controls inverter and charger.

ISA IVT Shunt

The ISA IVT-S shunt is the recommended pack-level monitor for all ZombieVerter builds. It provides continuous measurement of voltage, current, power, and state of charge over CAN.

What It Measures

Measurement	Spot value
Pack voltage (U1)	`udc`
DC current	`idc`
State of charge	`soc`
Power (W)	`power`
Coulomb count (Ah)	`curin`

Permanent 12V — The Critical Rule

■ The ISA shunt must have always-on 12V — the same permanent feed as the VCU itself.

If powered from an ignition-switched supply, the shunt loses its SoC state and UDC reading when the key is off. On the next startup, UDC reads zero → precharge fails every time.

U1 Placement — Inverter Side of Main Contactor

The ISA shunt's U1 voltage sense wire must connect to the HV bus on the **inverter side** of the main contactor — not the battery side.

During precharge, U1 rises from 0V toward pack voltage as the inverter capacitors charge through the precharge resistor. When U1 crosses `udcsw`, the VCU closes the main contactor.

If U1 is connected to the battery side, it reads full pack voltage immediately, and the VCU tries to close the main contactor before the capacitors are charged — potentially welding the contacts on first use.

Initialisation (One-Time)

```
shuntType = ISA
shuntCan = CAN1 (match your wiring)
```

```
→ ISAMode = Init → save to flash → power cycle VCU and shunt simultaneously
→ ISAMode = Normal → save to flash → reboot VCU
```

If initialisation fails, power the shunt 2–3 seconds before the VCU. This allows the shunt to fully boot before the VCU attempts to communicate.

BMW S-BOX Alternative

Set `shuntType` = SBOX. The BMW Safety Box provides pack voltage monitoring and HVIL (High Voltage Interlock Loop) in one unit. Popular in BMW-based builds. Configure in Comms parameters.

Cell-Level BMS Options

BMS	Notes
SimpBMS	Most popular. Open-source. Designed for EV conversions. Connects to Leaf cell modules via UART (120-pin Leaf BMS connector), sends charge/discharge limits to VCU over CAN.
Orion BMS	Commercial, professional-grade. Plug-and-play CAN integration with VCU. High cost but very reliable.
Nissan Leaf ZE1 BMS	Salvaged from 62kWh Leaf. Native Leaf cell integration. Supported in V2.30A.
Renault Kangoo BMS	36-module BMS. Added V2.20A.
BMW S-BOX	Combined pack monitor + HVIL. Set `shuntType` = SBOX.
VW E-BOX	Volkswagen equivalent to S-BOX. Set `shuntType` = VAG.

How BMS Limits Work

The BMS sends maximum charge and discharge current limits to the VCU via CAN. The VCU derates inverter torque commands and charge current to stay within those limits.

```
BMS sends via CAN → VCU enforces:
Max discharge current → caps motor torque demand
Max charge current → caps regen / charge current
BMS fault / over-volt → VCU opens contactors
```

BMS Parameters

Parameter	What it does
`bmsCan`	CAN bus the BMS is connected to
`bmsType`	BMS protocol (SimpBMS, Orion, ZE1, etc.)
`maxDischargeCurrent`	VCU hard limit (A) — overrides BMS if lower
`maxChargeCurrent`	VCU hard limit for regen and charging

■ **Set `bmsCan` correctly.** If BMS shares a bus with the inverter, check for CAN ID conflicts. The ISA shunt and cell BMS can be on different CAN buses — the ISA shunt often runs at 1 Mbps while most BMS devices run at 500

kbps, so they typically need separate buses anyway.

ISA vs Cell BMS — Both Needed

The ISA shunt and a cell BMS are complementary, not interchangeable:

- The **ISA shunt** provides UDC for precharge logic and idc for power monitoring. Without it (or `shuntType` = SBOX/VAG), precharge sequencing cannot work.
- The **cell BMS** provides per-cell voltage monitoring, temperature, balancing, and current limits. Without it, you have no protection against individual cell over-voltage or thermal runaway.

From V2.20A, it is possible to run with `shuntType` = None and use BMS data from the inverter and BMS directly for pack-level monitoring — demonstrated in Damien's Mitsubishi L200 truck build using a Leaf ZE1 battery pack with integrated BMS.

Source: openinverter.org/wiki · Damien Maguire @Evbmw

Last verified against firmware: V2.30A (August 2025)

TRACK ADVANCED · MODULE A04

Gear Selectors & Shifters

Overview

The VCU supports multiple methods of direction selection — from simple toggle switches to OEM electronic shifters communicating over CAN. The method is set via two parameters: `dirmode` (controls the 12V input logic) and `gearLever` (selects OEM CAN shifter protocol if used).

> **GS450h exception:** The GS450h has a physical P/R/N/D linkage connected to the transmission neutral safety switch. Note that with the exception of Park, which physically engages the parking pawl, the other positions are a simple electrical switch. The VCU reads switch state directly — it does not use the `gearLever` CAN parameter. See Module I03.

dirmode — Simple Direction Control

For builds using 12V switches or buttons rather than OEM CAN shifters.

dirmode	Wiring	Behaviour
`defaultFwd`	Pin 53 only	Auto-forward in run mode. Momentary 12V to Pin 53 = reverse. Release = back to forward.
`switch`	Pin 54 (Fwd) + Pin 53 (Rev)	3-position. Neither active = neutral. Best for PRND selector.
`button`	Single momentary	Press to toggle between forward and reverse.
`switchReversed`	Same as switch	Motor direction inverted — use when drivetrain spins backwards.
`buttonReversed`	Same as button	Toggle + inverted direction.

3-Position Switch (switch mode) — Recommended

```
D position → 12V to Pin 54 (din_forward = on)
N position → neither pin (no torque output)
R position → 12V to Pin 53 (din_reverse = on)
```

> ■ **Verify with Spot Values before road testing.** Sit with the web interface open. Move through each selector position. Confirm `din_forward` and `din_reverse` toggle correctly. A wiring error means unintended reverse at startup.

defaultFwd — Simplest Possible Build

Only needs one wire for reverse. Auto-Drive when in run mode. Single momentary button on Pin 53 for reverse. When released, returns to Drive. No PRND lever needed — ideal for minimal builds.

OEM CAN-Based Electronic Shifters

Modern vehicles use electronic shifters that send gear position over CAN. The gear shifter class (added V2.04A) reads these messages and maps them to the VCU's internal direction and Park/Neutral states.

Set the `gearLever` parameter to match your shifter:

Shifter	Added	Notes
12V inputs (default)	Always	Standard switch wiring — Pins 53/54
BMW F30 CAN e-shifter	V2.04A	Sends PRND via CAN. Clean OEM look. Reverse engineering by "project Gus."
JLR rotary shifter	V2.05A	Jaguar/Land Rover CAN rotary. Popular for custom builds.
BMW E65 7-series	V2.20A	Now uses shifter class (was hardcoded before V2.20A).

Check the openinverter wiki for the specific CAN ID required for your shifter model.

Park Gear (V2.05A+)

When Park is selected via an OEM shifter or a dedicated 12V input:

- **Torque inhibited** — no motor movement regardless of throttle position
- **HV bus maintained** — contactors stay closed, DC-DC converter active
- **Charge mode enabled** — some OBCs require Park to be selected before accepting a charge command

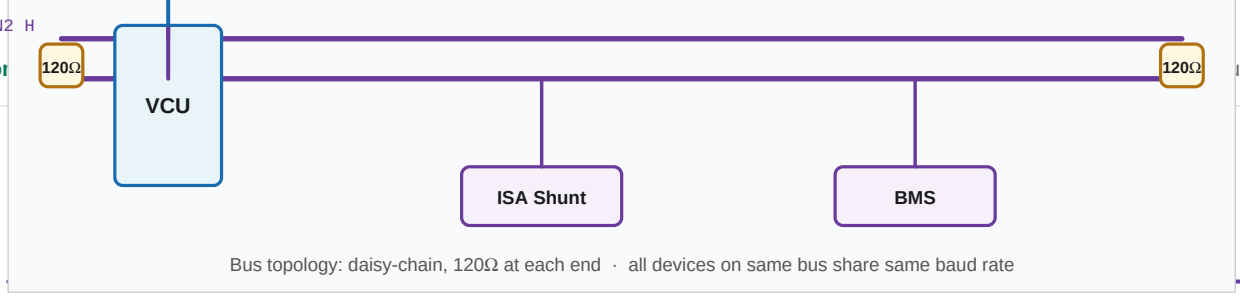
E91 Build Example

Jamie Jones (Ohm-Grown EVs) used the OEM BMW E91 automatic shifter assembly for his conversion. The shifter physically clicks into P/R/N/D positions but sends simple 12V signals — wired directly to VCU Pins 53/54. This is the simplest way to retain an OEM shifter feel without CAN complexity.

Source: Damien Maguire @Evbmw · Jamie Jones / Ohm-Grown EVs · openinverter.org

Gear shifter class added V2.04A · JLR added V2.05A · Park gear added V2.05A

Last verified against firmware: V2.30A (August 2025)



Bus topology: daisy-chain, 120Ω at each end · all devices on same bus share same baud rate

Overview

The ZombieVerter has three independent CAN buses. Understanding how to assign devices, terminate correctly, and integrate OEM instrument clusters is what separates a functional conversion from a polished one.

CAN Bus Physical Wiring

Termination — The Most Skipped Step

A CAN bus must have exactly **two 120Ω termination resistors** — one at each physical end of the bus. Without correct termination you get signal reflections, corrupted frames, and intermittent communication failures that are very hard to diagnose.

> ■ **Exactly two 120Ω resistors — one at each end of each bus.**

One resistor at the VCU end (often built into the VCU board). One at the furthest device on that bus. Between CAN H and CAN L.

> ■ **Check device datasheets.** Some inverters and OBCs have built-in 120Ω termination. Adding an external resistor to a device that already has one results in too many terminators and bus instability.

Topology — Bus Not Star

CAN is a bus topology. All devices connect to the same two wires (CAN H and CAN L) running from one end to the other. Avoid star topologies and long stub branches.

```
Correct — bus topology:
[120Ω] ■■■ VCU ■■■ ISA Shunt ■■■ Inverter ■■■ [120Ω]

Wrong — star topology:
VCU
■■■ ISA Shunt ← signal reflections at each stub
■■■ Inverter
■■■ BMS
```

Use twisted pair for all CAN runs. Shielded twisted pair for runs near HV cables or in noisy environments.

Baud Rates by Device

Device	Baud rate
Nissan Leaf inverter	500 kbps
Mitsubishi Outlander	500 kbps
ISA IVT shunt	1000 kbps (1 Mbps)
SimpBMS / Orion BMS	500 kbps

Device	Baud rate
BMW i3 LIM	500 kbps
CHAdeMO socket	500 kbps (fault-tolerant CAN3)
OBD2 / ELM327	500 kbps

All devices on the same CAN bus must run at the same baud rate. The ISA shunt at 1 Mbps typically needs its own bus separate from 500 kbps devices.

Assigning Devices to CAN Buses

The Three Buses

Bus	Notes
`CAN1`	High-speed CAN. Primary bus for most inverters.
`CAN2`	High-speed CAN. Secondary bus — BMS, shunt, second devices.
`CAN3`	Configurable via PCB jumpers — high-speed, fault-tolerant, or single-wire. Used for CHAdeMO.

Assignment Parameters

Parameter	What it assigns
`inverterCan`	Which bus the traction inverter is on
`shuntCan`	Which bus the ISA shunt is on
`bmsCan`	Which bus the cell BMS is on
`chargerCan`	Which bus the OBC is on

Typical Configuration

```
CAN1 @ 500 kbps:
Traction inverter (inverterCan = CAN1)
Leaf PDM (same bus as inverter)
BMW i3 LIM (CCS — same bus)

CAN2 @ 1000 kbps:
ISA shunt (shuntCan = CAN2)
SimpBMS / Orion (bmsCan = CAN2, if 500k: separate bus from shunt)

CAN3 @ 500 kbps fault-tolerant:
CHAdeMO socket (6 PCB jumpers required)
```

■ **CAN1/CAN2 label swap:** There is a known documentation issue where CAN1 and CAN2 are labelled in reverse in some older wiring diagrams. If a device won't communicate despite correct wiring, try swapping the bus assignment parameter value before rewiring.

Vehicle CAN Integration

The vehicle Parameter

Set `vehicle` to tell the VCU which OEM CAN profile to use. The VCU then sends spoofed CAN messages to keep the instrument cluster functional.

vehicle setting	Covers	What it drives
BMW E46	1998–2005 3-series	Speedo, tacho, temp, fuel level, CEL suppression
BMW E6x+	E60/E90/E9x/E91	Speedo, tacho, lock state, warning lights
BMW E39	1996–2003 5-series	Speedo, tacho, instrument functions
BMW E31	8-series	Added V2.04A
`VAG`	Mid-2000s VW/Audi	Instrument cluster CAN
`Subaru`	Various	CAN integration
`Classic`	Any vehicle	No vehicle CAN — uses analog outputs only
`None`	—	No vehicle CAN output

BMW E9x — Added V2.20A

The E90/E91/E92/E93 CAN integration added in V2.20A includes:

- Lock state over CAN
- Warning lights control (suppresses CEL and ICE-specific warnings)
- Vehicle speed driven from motor RPM

If Your Vehicle Isn't Listed

Use Classic mode and drive gauges with analog PWM outputs. You can also submit a CAN log from your OEM vehicle as a GitHub feature request — see Module A06.

Classic Mode & Analog Gauges

Set `vehicle` = Classic for any vehicle without CAN-based instruments — pre-2000 vehicles, classic cars, trucks, kit cars.

Tachometer — TachoPPR Parameter (V2.20A+)

The VCU outputs a pulse signal from a PWM output pin that can drive an OEM analog tachometer. `TachoPPR` sets pulses per revolution:

TachoPPR	Equivalent cylinder count
1	1-cylinder
2	4-cylinder (typical default)
3	6-cylinder
4	8-cylinder

Tune until the needle reads actual motor RPM. Assign the tacho output function in the Dout section.

Speedometer

A speedo pulse output is also available via the GS450h pump pin in V2.20A+ when set to Classic mode. Configure PPR to match your OEM speedo sender expectation.

Fuel Gauge — Digipot Method

For vehicles without CAN fuel gauge control, a digital potentiometer (digipot) IC wired between the VCU and the OEM fuel gauge sender wire can drive the fuel needle to show state of charge.

The VCU controls the digipot resistance in steps. Calibrate by setting step values and observing needle position:

- 0 steps = low fuel warning triggered
- Tune mid-scale and full-scale steps to match actual SoC at those points

The temperature gauge can also be driven from `temphs` (inverter heatsink temperature) or `tempm` (motor temperature) in some vehicle classes.

OBD2 & Live Telemetry

OBD2 via CAN (V2.04A+)

The VCU can respond to OBD2 requests over CAN, allowing standard Bluetooth OBD dongles and apps like Torque Pro to read live VCU data — motor speed, pack voltage, current, SoC, temperatures.

> ■ **Dongle quality matters.** Cheap AliExpress ELM327 clones have inconsistent results. Higher-quality Bluetooth or WiFi OBD adapters work reliably. If Torque Pro connects but shows no data, try a different dongle before assuming a configuration problem.

Web Interface Graphing

The VCU web interface includes built-in real-time graphing. Select spot values to plot, set the update interval. Useful for:

- Throttle linearity during calibration (plot `potnom` vs `pot`)
- Precharge voltage rise (plot `udc` over time)
- Temperature monitoring under load (`temphs`, `tempm`)
- Regen tuning (`idc` going negative during deceleration)

Parameter Files on GitHub

Damien publishes parameter files for his fleet vehicles (E39, E46, E31, L200 truck) on the ZombieVerter GitHub repo. These are real working configurations for tested vehicles — a useful starting point for matching drivetrain/vehicle combinations before tuning your own build.

Diagnosing CAN Communication Failures

Work through in order:

1. Is the correct bus assignment parameter set?
2. Is the baud rate consistent across all devices on that bus?
3. Is termination correct? Measure ~60Ω across H/L with power off.
4. Try swapping `inverterCan` or `shuntCan` value (CAN1/CAN2 label swap issue).
5. Are CAN H and CAN L wires the right way around? Try swapping them.
6. Is the cable twisted pair? Long untwisted runs in noisy environments cause intermittent failures.

Source: Damien Maguire @Evbmw · openinverter.org/wiki · V2.04A changelog · V2.20A changelog

TachoPPR added V2.20A · E90 CAN added V2.20A · OBD2 added V2.04A · E31 added V2.04A

Last verified against firmware: V2.30A (August 2025)

TRACK ADVANCED · MODULE A06

Firmware Customisation & Contributing

Overview

The ZombieVerter firmware is fully open source, hosted at github.com/damienmaguire/Stm32-vcu. When you need something the firmware doesn't yet support, you have three paths — and one of them requires no coding at all.

Path 1 — GitHub Feature Request (No Coding Required)

The simplest path. File an issue on GitHub describing what you need. The maintainers and community evaluate it, and if it fits the project goals and has sufficient demand, it gets implemented.

How to File a Good Request

Search existing issues first. Your feature may already have been requested, discussed, or even implemented in a recent release that you haven't updated to yet. Check both open and closed issues.

Describe the use case, not the implementation. "I need to control a Renault Zoe OBC" is a better issue than "add parameter X with value Y to CAN message Z." The maintainers understand the codebase far better than most requesters — explain the problem you're trying to solve.

Provide CAN logs if relevant. For new inverter or charger support, a CAN log captured from the OEM vehicle is the single most valuable thing you can provide. Without it, reverse engineering has to start from zero. This is why the forum is full of "does anyone have a CAN log for X?" threads — they unlock new hardware support.

Post on the forum first. openinverter.org/forum is the right first stop before GitHub. If others need the same feature, the forum thread becomes evidence of demand. It also filters out requests that are already supported with a simple parameter change.

What Gets Prioritised

Priority	Type of request
High	Bug fixes · Safety issues · Widely-used hardware with CAN log data available
Medium	New inverter/charger support with community demand · Features partially implemented
Lower	Niche hardware with single user · Features requiring significant reverse engineering from scratch

Path 2 — Fork and Self-Implement

Clone the repo, make your changes, compile, flash to your VCU, and run it. You're entirely in control. No obligation to contribute back to the main project — it's your fork.

```
# Clone with all submodules — this step is critical
git clone --recurse-submodules git@github.com:damienmaguire/Stm32-vcu.git
```

■ `--recurse-submodules` **is mandatory.** The firmware depends on `libopeninv` (Johannes Huebner's openinverter framework) and other submodules. Without this flag the build fails with missing includes.

Build Environment

The firmware uses an ARM GCC toolchain. Build documentation is on the openinverter.org wiki. Johannes Huebner's GitHub (github.com/jsphuebner) is the reference for the underlying parameter system and CAN class architecture — understanding his framework is essential for meaningful customisation.

GD32 Boards

■ **If you have a GD32F107 board, the GD_Zombie branch was abandoned and has substantially diverged from main.** Self-building for a GD board means starting from that old branch — you will not have features from V2.00A onwards. See Module C01 for chip identification.

Path 3 — Pull Request to Main Branch

If you implement something genuinely useful for the wider community, you can submit a pull request. This is how most new hardware support, vehicle classes, and firmware features have landed — contributions from community members working on their own builds who then share the work.

Examples of community contributions that made it into main: BMW E31 vehicle class, OBD2 support, analog IO matrix, JLR gear shifter, Kangoo BMS, MG ZS OBC.

The Testing Requirement

■ **Changes must be tested in real vehicles before merging. This is non-negotiable.**

The firmware runs high-voltage systems in moving vehicles. A bug that gets through to a release can cause a precharge failure at speed, unintended motor torque, a contactor fault, or worse. Damien personally tests new features across multiple vehicles — typically 3–4 different builds covering different drivetrain and charger combinations — before any release goes out.

This is why release cycles are measured in months, not weeks. V2.20A was tested across four running vehicles (BMW E39, E46, E31, and the Mitsubishi L200 truck) before release. The testing isn't just "does it compile and power up" — it's sustained real-world driving with the full complement of features active.

What a Good Pull Request Includes

- Working, clean code
- Evidence of testing in at least one real vehicle (ideally more)
- CAN log data if it's a new device integration
- A description of what was tested, what edge cases were considered, and any known limitations
- No breaking changes to existing functionality without discussion first

The community is welcoming to well-tested contributions. The project's growth from a single-vehicle GS450h controller to a universal VCU supporting dozens of drivetrains and chargers was built on exactly this kind of contribution.

The Openinverter Framework

The ZombieVerter firmware is built on Johannes Huebner's openinverter firmware framework (`libopeninv`). Understanding the framework is important for anyone doing serious firmware work:

- **Parameter system** — all configurable parameters are defined in a structured way that automatically exposes them in the web interface
- **CAN class architecture** — each device (inverter, charger, BMS) is implemented as a CAN class that can be selected at runtime via the `inverter` / `chargerType` parameters
- **IO Matrix** — the flexible input/output assignment system introduced in V2.00A

Johannes's GitHub and the openinverter forum are the best resources for understanding the framework internals.

Source: github.com/damienmaguire/Stm32-vcu · openinverter.org/forum · Damien Maguire @Evbmw

Last verified against firmware: V2.30A (August 2025)